

STAT 5124: Linear Models Theory

Chapter 6: Optimization

Yili Hong

Section 6.1: Introduction

Section 6.2: Gradient-Based Methods

Section 6.5: Convex Optimization and ADMM

Section 6.6 Coordinate Descent

Outline

Overview of Optimization Techniques

Gradient-Based Methods

- Gradient Descent

- Stochastic Gradient Descent

Convex Optimization

- Alternating Direction Method of Multipliers (ADMM)

Coordinate Descent Method

Optimization for Model Fitting

- ▶ Models depend on unknown parameters $\theta \in \mathbb{R}^p$.
- ▶ Model fitting is often formulated as an optimization problem.
- ▶ Goal: choose θ to minimize a loss function based on the data.

$$\min_{\theta \in \mathbb{R}^p} \ell(\theta | \text{Data}) \quad (1)$$

subject to some constraints.

- ▶ $\ell(\theta)$ measures model error (loss) given the data.
- ▶ Maximization problems can be written as minimizing $-\ell(\theta)$.

Components of an Optimization Problem

- ▶ A general optimization problem has three components:
 - ▶ Decision variable θ (model parameters)
 - ▶ Objective function $\ell(\theta)$ (loss function)
 - ▶ Constraint set
- ▶ The constraint set may be empty (no constraints on θ).
- ▶ Many statistical methods fit this framework, e.g.,
 - ▶ Least-squares estimation
 - ▶ Maximum likelihood estimation

Examples

1. For the Lagrange multiplier technique, the objective function to be minimized is $f(\mathbf{x})$, the decision variable is $\mathbf{x}$, and the constraints set is $\mathbf{h}(\mathbf{x}) = \mathbf{0}$.
2. For the least-squares (LS) problem with a linear model, the objective function is $\ell(\beta)$, and the decision variable is β with no constraints.
3. For the MLE problem, the objective function can be taken as the negative of the log-likelihood function, with $\theta = (\beta^\top, \sigma^2)^\top$ as the decision variable. That is, we define the loss function as $\ell(\theta) = -\log(L(\beta, \sigma^2))$.
4. Another example of a constrained optimization is in the proof of Theorem 5.1, where maximizing the likelihood under the null hypothesis is equivalent to minimizing $\ell(\theta) = -\log(L(\beta, \sigma^2))$ subject to the constraints $C\beta = \mathbf{t}$.

Numerical Optimization Algorithms

- ▶ Numerical optimization algorithms update parameters iteratively.
- ▶ Let θ_k be the parameter estimate at step k .

$$\theta_{k+1} = m(\theta_k, \ell(\theta_k))$$

- ▶ $m(\cdot, \cdot)$ determines how the parameter update is performed.
- ▶ Three main types of optimization methods:
 - ▶ Gradient-based (first-order derivatives)
 - ▶ Hessian-based (second-order derivatives)
 - ▶ Function-value based methods

Gradient-Based Optimization

- ▶ Key idea: compute the gradient of the loss with respect to the parameter vector.
- ▶ The gradient indicates the direction of steepest increase of the loss.
- ▶ Optimization algorithms use the gradient to determine the next update step.
- ▶ Common gradient-based methods:
 - ▶ Gradient Descent
 - ▶ Stochastic Gradient Descent (SGD)

Hessian-Based Optimization

- ▶ Hessian-based methods (Newton methods) use second derivatives.
- ▶ The matrix of second derivatives is called the Hessian matrix.
- ▶ The Hessian helps determine an appropriate step size for updating parameters.
- ▶ Advantage: can provide faster convergence near the optimum.
- ▶ Tradeoff: computing second derivatives can be computationally expensive.
- ▶ In practice, numerical derivatives are often used for both gradient and Hessian methods.

Function-Value Based Optimization

- ▶ Numerical derivatives (finite differences) may introduce approximation errors.
- ▶ Function-value based methods avoid derivatives and use only objective function values.
- ▶ Example: Nelder–Mead method.
- ▶ Typically slower than gradient-based methods, but often more robust.
- ▶ Can be applied to non-differentiable objective functions.

Gradient-Based Methods

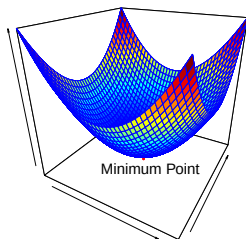
- ▶ For an objective function $\ell(\theta)$, the *gradient* with respect to its argument θ is defined as

$$\nabla \ell(\theta) = \frac{\partial \ell(\theta)}{\partial \theta}. \quad (2)$$

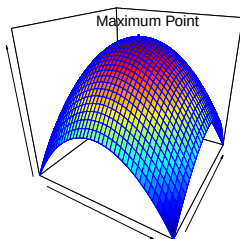
- ▶ A *stationary point* (or *critical point*) of an objective function $\ell(\theta)$ is a point where the gradient is zero. That is, it is a solution to the equations

$$\nabla \ell(\theta) = \mathbf{0}.$$

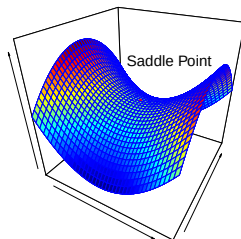
Illustration



(a) Minimum point



(b) Maximum point



(c) Saddle point

Figure: Illustration of different types of stationary points.

More Definitions

- ▶ The *Hessian matrix* for the loss function $\ell(\boldsymbol{\theta})$ is

$$\nabla^2 \ell(\boldsymbol{\theta}) = \frac{\partial^2 \ell(\boldsymbol{\theta})}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}^\top}.$$

- ▶ A symmetric matrix A is said to be
 1. *negative definite* if $(-A)$ is positive definite.
 2. *negative semi-definite* if $(-A)$ is positive semi-definite.
 3. *indefinite* if it does not belong to any of the following:
positive definite, negative definite, positive semi-definite,
negative semi-definite, and a zero matrix.

Theoretical Results

Let θ_0 be a stationary point for the objective function $\ell(\theta)$, that is, $\nabla\ell(\theta_0) = \mathbf{0}$. Then

1. θ_0 is a local minimum (maximum) of $\ell(\theta)$ if $\nabla^2\ell(\theta_0)$ is positive (negative) definite.
2. θ_0 is a saddle point, if $\nabla^2\ell(\theta_0)$ is indefinite.
3. θ_0 can be a local minimum (maximum) or a saddle point, if $\nabla^2\ell(\theta_0)$ is positive (negative) semi-definite.

Example: Least Squares Loss and Derivatives

- ▶ Least-squares loss for the linear model:

$$\ell(\beta) = \frac{1}{2} \|\mathbf{y} - X\beta\|_2^2$$

- ▶ Gradient:

$$\frac{\partial \ell(\beta)}{\partial \beta} = X^\top X\beta - X^\top \mathbf{y}$$

- ▶ Setting the gradient to zero gives the normal equations:

$$X^\top X\beta = X^\top \mathbf{y}$$

- ▶ Hessian matrix:

$$\nabla^2 \ell(\beta) = X^\top X$$

Stationary Points of the LS Loss

- ▶ If X has full column rank, then $X^\top X$ is positive definite.
 - ▶ The LS equations have a unique solution.
 - ▶ There is a single stationary point of $\ell(\beta)$.
 - ▶ This stationary point is at least a local minimum.
- ▶ If X does not have full column rank, then $X^\top X$ is positive semi-definite.
 - ▶ The LS equations have multiple solutions.
 - ▶ Hence $\ell(\beta)$ has multiple stationary points.
 - ▶ Each solution is a local minimum (not a saddle point).
- ▶ Whether these are global minima will be discussed later in convex optimization.

Gradient Descent

- ▶ Goal: minimize $\ell(\boldsymbol{\theta})$.
- ▶ The gradient $\nabla\ell(\boldsymbol{\theta})$ indicates the direction of steepest increase of the loss.
- ▶ To reduce the loss, move in the opposite direction of the gradient.

Gradient descent update at iteration k :

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \delta \nabla\ell(\boldsymbol{\theta}_k)$$

- ▶ $-\nabla\ell(\boldsymbol{\theta}_k)$: direction of steepest descent.
- ▶ δ : learning rate (step size).
- ▶ Stop when improvement in the loss is very small:

$$|\ell(\boldsymbol{\theta}_{k+1}) - \ell(\boldsymbol{\theta}_k)| \leq \tau.$$

Illustration

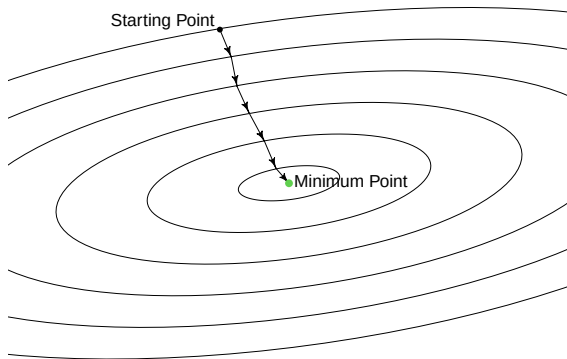


Figure: Illustration of gradient descent in finding a minimum point.

Key Idea Behind Gradient Descent

- ▶ Gradient descent requires computing the gradient $\nabla \ell(\boldsymbol{\theta})$.
- ▶ Gradients may be computed
 - ▶ exactly, or
 - ▶ numerically (finite-difference approximation).
- ▶ The gradient indicates a direction that decreases the loss.
- ▶ Intuition: move slightly in the negative gradient direction to reduce $\ell(\boldsymbol{\theta})$.

Key result:

If $\|\nabla \ell(\boldsymbol{\theta})\|_2 \neq 0$, there exists a sufficiently small step size δ such that

$$\ell(\boldsymbol{\theta}_{k+1}) < \ell(\boldsymbol{\theta}_k).$$

Choosing the Learning Rate in Gradient Descent

- ▶ The learning rate δ controls the step size in gradient descent.
- ▶ If δ is too large:
 - ▶ The update may overshoot the minimum.
 - ▶ The algorithm may move further away from the optimum.
- ▶ If δ is too small:
 - ▶ Convergence becomes very slow.
 - ▶ Practical starting values are often 0.01 or 0.001.
- ▶ The learning rate can vary with iteration (δ_k), called *adaptive gradient descent*.
- ▶ Gradient descent finds a stationary point, which may be a global minimum, local minimum, or saddle point.

Motivation for Stochastic Gradient Descent

- ▶ In many data modeling problems, the loss function is a sum over observations:

$$\ell(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \ell_i(\boldsymbol{\theta}).$$

- ▶ The gradient is the average of individual gradients:

$$\nabla \ell(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \nabla \ell_i(\boldsymbol{\theta}).$$

- ▶ When n is large, computing the full gradient can be expensive in time and memory.
- ▶ *Stochastic gradient descent (SGD)* addresses this issue by updating parameters using only a small subset of the data.

Stochastic Gradient Descent (SGD)

- ▶ Instead of using all n observations, SGD uses a random sample of size b .
- ▶ The sampled observations form a *mini-batch* $\mathcal{B}$.
- ▶ The gradient is approximated by

$$\nabla \ell(\boldsymbol{\theta}) \approx \frac{1}{b} \sum_{i \in \mathcal{B}} \nabla \ell_i(\boldsymbol{\theta}).$$

- ▶ Each iteration uses a new randomly selected mini-batch.
- ▶ Small b reduces computation time per iteration.
- ▶ Tradeoff: noisier gradient $\Rightarrow$ more iterations may be needed.

Illustration

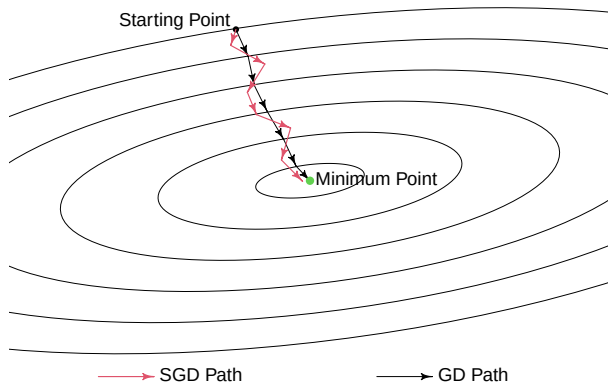


Figure: Illustration of the path of SGD with a comparison with the path from GD.

Multiple Minimum Points

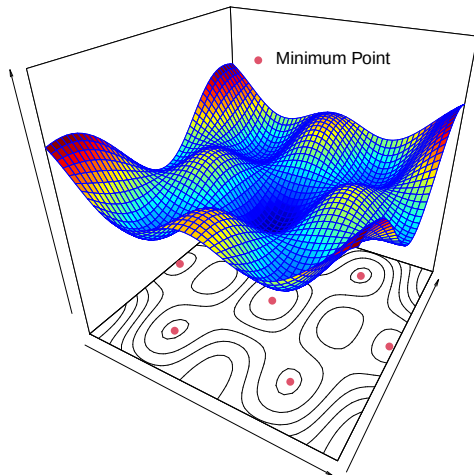


Figure: Illustration of multiple minimum points.

SGD and the Adam Algorithm

- ▶ Stochastic gradient descent (SGD) is well suited for large datasets:
 - ▶ reduces computing time per iteration
 - ▶ lowers memory requirements
- ▶ GD/SGD may get stuck at a local minimum when the loss surface has multiple minima.
- ▶ Two common strategies to improve optimization:
 - ▶ momentum (uses past gradients)
 - ▶ adaptive learning rates
- ▶ The **Adam algorithm** (Adaptive Moments) combines these ideas.
- ▶ Adam adapts the learning rate using the *first and second moments* of the stochastic gradient.

Example: Griewank Function

- ▶ Consider the 2D Griewank function

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^2 \frac{\theta_i^2}{30} - \prod_{i=1}^2 \cos\left(\frac{\theta_i}{\sqrt{i}}\right) + 1, \quad \boldsymbol{\theta} = (\theta_1, \theta_2)^\top.$$

- ▶ The function has multiple stationary points and a unique global minimum.
- ▶ Used to illustrate behavior of optimization algorithms.
- ▶ In this example:
 - ▶ GD: learning rate $\delta = 0.01$.
 - ▶ Adam: $\delta = 0.01$, $\rho_1 = 0.9$, $\rho_2 = 0.925$.
- ▶ GD may get trapped in a local minimum depending on the starting point.
- ▶ Adam can escape local minima with appropriate tuning parameters.

Result Visualization

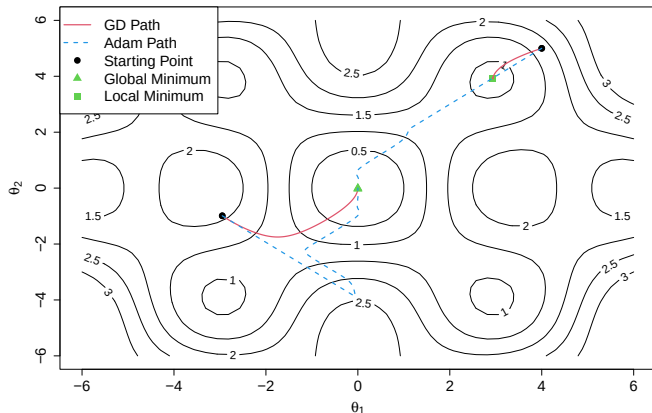


Figure: Illustration of the iteration paths for the GD and Adam algorithm in finding minimum. The contour plots shows the Griewank function.

Convex Sets and Convex Functions

- ▶ Convexity is an important property in optimization.
- ▶ For a convex function, every local minimum is also a global minimum.
- ▶ Example: the least-squares loss function is convex.

Convex set:

A set $\mathcal{S} \subseteq \mathbb{R}^p$ is convex if for any $\mathbf{x}_1, \mathbf{x}_2 \in \mathcal{S}$,

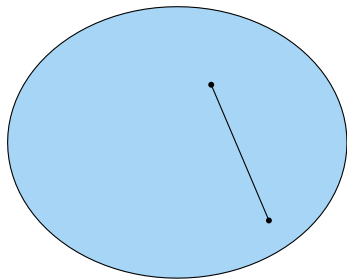
$$\mathbf{x} = \alpha \mathbf{x}_1 + (1 - \alpha) \mathbf{x}_2 \in \mathcal{S}, \quad \alpha \in (0, 1).$$

Convex function:

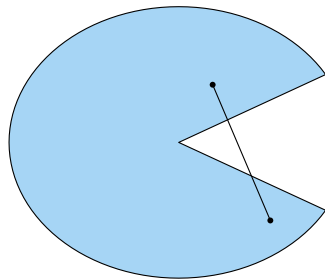
A function $f(\mathbf{x})$ on a convex set $\mathcal{S}$ is convex if

$$f(\alpha \mathbf{x}_1 + (1 - \alpha) \mathbf{x}_2) \leq \alpha f(\mathbf{x}_1) + (1 - \alpha) f(\mathbf{x}_2).$$

Illustration



(a) Convex Set



(b) Non-convex Set

Figure: Examples of convex and non-convex sets.

Properties of Convex Functions

- ▶ Convex functions have several useful properties in optimization.
- ▶ **Key properties:**
 - ▶ The sum of convex functions is convex.
 - ▶ The sum of a convex function and a strictly convex function is strictly convex.
 - ▶ For a convex function, every local minimum is a global minimum.
 - ▶ A strictly convex function has at most one stationary point, which (if it exists) is the global minimum.

Convexity and Least Squares

- ▶ The LS loss function is a convex function.
- ▶ By convexity, every local minimum of the LS loss is also a global minimum.
- ▶ Therefore, any solution to the LS equations is a global minimum of the loss function.
- ▶ If X has full column rank:
 - ▶ the LS solution is unique,
 - ▶ and it is the only point achieving the global minimum.

Alternating Direction Method of Multipliers (ADMM)

- ▶ Convex optimization can benefit from specialized algorithms that exploit convexity.
- ▶ One important method is the **alternating direction method of multipliers (ADMM)**.
- ▶ ADMM decomposes a large optimization problem into smaller subproblems.
- ▶ This decomposition can greatly improve computational efficiency for large-scale problems.
- ▶ In data modeling, ADMM is used in problems such as:
 - ▶ LASSO and other penalized regression methods
 - ▶ Regularized generalized linear models

ADMM Optimization Formulation

- ▶ ADMM is designed for constrained convex optimization problems of the form:

$$\begin{array}{ll}\min & f(\mathbf{x}) + g(\mathbf{z}) \\ \text{s.t.} & A\mathbf{x} + B\mathbf{z} = \mathbf{c}\end{array}$$

- ▶ Variables: $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{z} \in \mathbb{R}^m$.
- ▶ $f(\cdot)$ and $g(\cdot)$ are convex (not necessarily smooth).
- ▶ The constraint couples the variables through $A\mathbf{x} + B\mathbf{z} = \mathbf{c}$.
- ▶ Such formulations arise in many statistics and machine learning problems.

LASSO Formulation for ADMM

- ▶ The LASSO optimization problem can be written as

$$\begin{aligned} \min \quad & \frac{1}{2} \|\mathbf{y} - X\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_1 \\ \text{s.t.} \quad & \boldsymbol{\beta} = \boldsymbol{\theta}. \end{aligned}$$

- ▶ In the ADMM form:

- ▶ $\mathbf{x} = \boldsymbol{\beta}, \quad \mathbf{z} = \boldsymbol{\theta}$
- ▶ $f(\boldsymbol{\beta}) = \frac{1}{2} \|\mathbf{y} - X\boldsymbol{\beta}\|_2^2$
- ▶ $g(\boldsymbol{\theta}) = \lambda \|\boldsymbol{\theta}\|_1$
- ▶ $A = I, \quad B = -I, \quad \mathbf{c} = \mathbf{0}$

- ▶ The constraint $\boldsymbol{\beta} = \boldsymbol{\theta}$ separates the quadratic loss and the L_1 penalty, enabling ADMM updates.

The ADMM Algorithm

- ▶ Initialize $\mathbf{z}_0$, $\mathbf{u}_0$, and choose a constant ρ .
- ▶ At iteration k :
 - ▶ **x-update:**

$$\mathbf{x}_{k+1} = \underset{\mathbf{x}}{\operatorname{argmin}} f(\mathbf{x}) + \frac{\rho}{2} \|\mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{z}_k - \mathbf{c} + \mathbf{u}_k\|_2^2$$

- ▶ **z-update:**

$$\mathbf{z}_{k+1} = \underset{\mathbf{z}}{\operatorname{argmin}} g(\mathbf{z}) + \frac{\rho}{2} \|\mathbf{A}\mathbf{x}_{k+1} + \mathbf{B}\mathbf{z} - \mathbf{c} + \mathbf{u}_k\|_2^2$$

- ▶ **u-update:**

$$\mathbf{u}_{k+1} = \mathbf{u}_k + \mathbf{A}\mathbf{x}_{k+1} + \mathbf{B}\mathbf{z}_{k+1} - \mathbf{c}$$

- ▶ Repeat until convergence.

Example: Distributed Least Squares via ADMM

- ▶ For very large datasets, split the data into subsets and fit a common model.
- ▶ Example: divide data into two parts.

$$\ell(\beta) = \frac{1}{2} \|\mathbf{y} - X\beta\|_2^2 = \frac{1}{2} \|\mathbf{y}_1 - X_1\beta\|_2^2 + \frac{1}{2} \|\mathbf{y}_2 - X_2\beta\|_2^2$$

- ▶ Write the problem in ADMM form:

$$\begin{array}{ll} \min & f(\mathbf{x}) + g(\mathbf{z}) \\ \text{s.t.} & \mathbf{x} - \mathbf{z} = \mathbf{0} \end{array}$$

- ▶ where

$$f(\mathbf{x}) = \frac{1}{2} \|\mathbf{y}_1 - X_1\mathbf{x}\|_2^2, \quad g(\mathbf{z}) = \frac{1}{2} \|\mathbf{y}_2 - X_2\mathbf{z}\|_2^2.$$

ADMM Updates for Distributed OLS

ADMM objective:

$$\ell_{\rho}(\mathbf{x}, \mathbf{z}, \mathbf{u}) = f(\mathbf{x}) + g(\mathbf{z}) + \frac{\rho}{2} \|\mathbf{x} - \mathbf{z} + \mathbf{u}\|_2^2 - \frac{\rho}{2} \|\mathbf{u}\|_2^2.$$

Iterative updates:

$$\mathbf{x}_{k+1} = (X_1^{\top} X_1 + \rho I)^{-1} (X_1^{\top} \mathbf{y}_1 - \rho(\mathbf{z}_k - \mathbf{u}_k))$$

$$\mathbf{z}_{k+1} = (X_2^{\top} X_2 + \rho I)^{-1} (X_2^{\top} \mathbf{y}_2 - \rho(\mathbf{x}_{k+1} - \mathbf{u}_k))$$

$$\mathbf{u}_{k+1} = \mathbf{u}_k + \mathbf{x}_{k+1} - \mathbf{z}_{k+1}.$$

- Enables distributed computation of OLS for large datasets.

Coordinate Descent (CD)

- ▶ It is an **approach** to optimization, not a specific algorithm.
- ▶ Optimize the objective function one coordinate at a time.

$$\ell(\boldsymbol{\theta}) = \ell(\theta_1, \theta_2, \dots, \theta_p)$$

- ▶ Update coordinates sequentially:

$$\hat{\theta}_1 = \operatorname{argmin}_{\theta} \ell(\theta, \theta_2, \dots, \theta_p)$$

$$\hat{\theta}_2 = \operatorname{argmin}_{\theta} \ell(\hat{\theta}_1, \theta, \theta_3, \dots, \theta_p)$$

- ▶ Continue until $\hat{\theta}_p$, completing one cycle.
- ▶ Repeat cycles until convergence.
- ▶ Each 1-dim optimization can use any suitable method.

Illustration

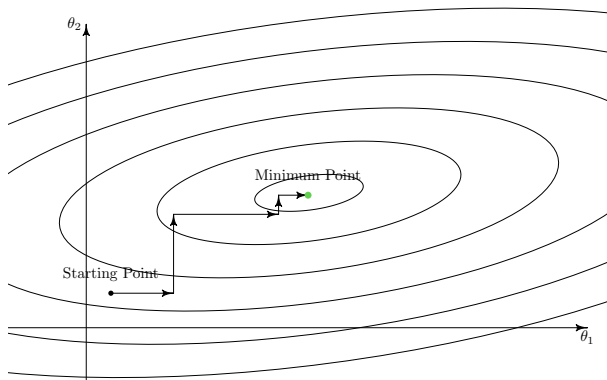


Figure: An illustration of the coordinate descent algorithm.

Advantages of Coordinate Descent

- ▶ Coordinate descent (CD) is a **strategy** for applying optimization methods.
- ▶ It does not specify how the one-dimensional optimizations are solved.
- ▶ Key idea: optimize parameters one at a time rather than jointly.
- ▶ Advantages:
 - ▶ Simpler 1-dimensional optimizations
 - ▶ Scales well to high-dimensional problems
 - ▶ Works with non-differentiable or discontinuous functions
- ▶ If available, closed-form updates can be used; otherwise numerical methods are applied.

Coordinate Descent for Linear Models

- ▶ Consider the linear model $\mathbf{y} = X\beta + \epsilon$ with LS loss.
- ▶ Coordinate descent updates parameters sequentially:

$$\beta_0 \rightarrow \beta_1 \rightarrow \cdots \rightarrow \beta_k.$$

- ▶ Let $\mathbf{x}_j$ be the column of X corresponding to β_j .
- ▶ The update for β_j is

$$\beta_j \leftarrow \frac{\epsilon_{(j)}^\top \mathbf{x}_j}{\mathbf{x}_j^\top \mathbf{x}_j},$$

where $\epsilon_{(j)} = \mathbf{y} - X\beta_{(j)}$ is the residual with $\beta_j = 0$.

- ▶ Advantages:
 - ▶ No inversion of $X^\top X$
 - ▶ Efficient when p is large
 - ▶ $\mathbf{x}_j^\top \mathbf{x}_j$ can be precomputed

STAT 5124: Linear Models Theory

Chapter 8: Regularization

Yili Hong

Section 8.2: Ridge Regression

Outline

Setup and Motivation

Theoretical Foundations of Ridge Regression

Selection of λ with Cross Validation

The Bayesian Connection

Basic Setup

- ▶ Consider the standard linear model with design matrix $X_{n \times p}$ and response vector $\mathbf{y}$:

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\epsilon} \quad (1)$$

where $\mathbb{E}(\boldsymbol{\epsilon}) = \mathbf{0}$ and $\text{Cov}(\boldsymbol{\epsilon}) = \sigma^2 I$.

- ▶ For inference procedures (prediction, interval estimation, hypothesis testing), further assume that $\boldsymbol{\epsilon} \sim N(\mathbf{0}, \sigma^2 I)$.

Motivation

- ▶ If X has full column rank p , when we have **multicollinearity**, some eigenvalues of $X^T X$ are very small or even zero.
- ▶ To address **multicollinearity**, **ridge regression** was introduced.
- ▶ In this situation, adding a small, positive perturbation to the diagonal elements of $X^T X$ can move all of its eigenvalues sufficiently away from zero to overcome problems due to the multicollinearity.

Loss Function

- ▶ The *ridge regression estimator* of β minimizes the loss function

$$\ell(\beta, \lambda) = \frac{1}{2} \|\mathbf{y} - X\beta\|_2^2 + \frac{\lambda}{2} \|\beta\|_2^2 \quad (2)$$

with respect to β , where the tuning parameter λ is some nonnegative constant.

Loss Function: Key Takeaways

- ▶ The tuning parameter λ is some nonnegative constant.
- ▶ $\|\beta\|_2^2 = \sum_{j=1}^p \beta_j^2$ is the square of the L_2 norm of β .
- ▶ $\lambda\|\beta\|_2^2/2$ is the ridge penalty.

Loss Function: Another Perspective

- ▶ Minimizing the penalized least squares $\ell(\beta, \lambda)$ also solves this constrained least squares problem:

$$\min_{\beta} \frac{1}{2} \|\mathbf{y} - X\beta\|_2^2 \quad \text{s.t.} \quad \|\beta\|_2^2 \leq c, \quad (3)$$

- ▶ For example, the R package `glmnet` uses

$$\ell(\beta, \lambda) = \frac{1}{2n} \|\mathbf{y} - X\beta\|_2^2 + \frac{\lambda}{2} \|\beta\|_2^2. \quad (4)$$

The Ridge Regression Estimator

- ▶ To obtain the ridge estimator of β , simply differentiate $\ell(\beta, \lambda)$ with respect to β and set the result to zero.

$$\frac{\partial \ell(\beta, \lambda)}{\partial \beta} = (-X^\top \mathbf{y} + X^\top X \beta) + \lambda I \beta \stackrel{\text{set}}{=} \mathbf{0},$$

which yields the equations

$$(X^\top X + \lambda I) \beta = X^\top \mathbf{y}.$$

For any $\lambda > 0$, $(X^\top X + \lambda I)$ is positive definite and thus nonsingular, regardless of the values of n , p , and $\text{rank}(X)$.

- ▶ The **solution** is

$$\hat{\beta}_{\text{RDG}} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}. \quad (5)$$

Properties of the Ridge Regression Estimator

The ridge estimator (5) of β for model (7) is unique given $\lambda > 0$. Moreover, the **mean** and **covariance matrix** of this estimator are

$$\begin{aligned}\mathbb{E}(\hat{\beta}_{\text{RDG}}) &= (X^{\top}X + \lambda I)^{-1}X^{\top}X\beta \neq \beta \text{ and} \\ \text{Cov}(\hat{\beta}_{\text{RDG}}) &= \sigma^2(X^{\top}X + \lambda I)^{-1}X^{\top}X(X^{\top}X + \lambda I)^{-1}.\end{aligned}$$

MSE of a Vector Estimator

The **mean square error (mse)** of a vector estimator $\hat{\boldsymbol{\theta}}_{p \times 1}$ is defined as the average squared distance of $\hat{\boldsymbol{\theta}}$ from the target parameter $\boldsymbol{\theta}$:

$$\text{mse}(\hat{\boldsymbol{\theta}}) = \mathbb{E} \|\hat{\boldsymbol{\theta}} - \boldsymbol{\theta}\|_2^2 = \mathbb{E}((\hat{\boldsymbol{\theta}} - \boldsymbol{\theta})^\top (\hat{\boldsymbol{\theta}} - \boldsymbol{\theta})).$$

It is a direct assessment of the quality of the estimator.

MSE of a Vector Estimator

The **mse** of a vector estimator $\hat{\boldsymbol{\theta}}_{p \times 1}$ can be computed as

$$\begin{aligned}\text{mse}(\hat{\boldsymbol{\theta}}) &= \text{tr}(\text{Cov}(\hat{\boldsymbol{\theta}})) + \|\mathbb{E}(\hat{\boldsymbol{\theta}}) - \boldsymbol{\theta}\|_2^2 \\ &= \sum_{i=1}^p (\text{Var}(\hat{\theta}_i) + \text{Bias}^2(\hat{\theta}_i))\end{aligned}$$

MSE of a Vector Estimator

Proof.

Expand the expression to give

$$\begin{aligned}\text{MSE}(\hat{\boldsymbol{\theta}}) &= \mathbb{E}((\hat{\boldsymbol{\theta}} - \mathbb{E}(\hat{\boldsymbol{\theta}}) + \mathbb{E}(\hat{\boldsymbol{\theta}}) - \boldsymbol{\theta})^\top (\hat{\boldsymbol{\theta}} - \mathbb{E}(\hat{\boldsymbol{\theta}}) + \mathbb{E}(\hat{\boldsymbol{\theta}}) - \boldsymbol{\theta})) \\&= \mathbb{E}((\hat{\boldsymbol{\theta}} - \mathbb{E}(\hat{\boldsymbol{\theta}}))^\top (\hat{\boldsymbol{\theta}} - \mathbb{E}(\boldsymbol{\theta}))) + (\mathbb{E}(\hat{\boldsymbol{\theta}}) - \boldsymbol{\theta})^\top (\mathbb{E}(\hat{\boldsymbol{\theta}}) - \boldsymbol{\theta}) \\&= \text{tr}(\mathbb{E}((\hat{\boldsymbol{\theta}} - \mathbb{E}(\hat{\boldsymbol{\theta}}))^\top (\hat{\boldsymbol{\theta}} - \mathbb{E}(\boldsymbol{\theta})))) + \|\mathbb{E}(\hat{\boldsymbol{\theta}}) - \boldsymbol{\theta}\|_2^2 \\&= \text{tr}(\mathbb{E}((\hat{\boldsymbol{\theta}} - \mathbb{E}(\hat{\boldsymbol{\theta}}))(\hat{\boldsymbol{\theta}} - \mathbb{E}(\boldsymbol{\theta}))^\top)) + \|\mathbb{E}(\hat{\boldsymbol{\theta}}) - \boldsymbol{\theta}\|_2^2 \\&= \text{tr}(\text{Cov}(\boldsymbol{\theta})) + \|\mathbb{E}(\hat{\boldsymbol{\theta}}) - \boldsymbol{\theta}\|_2^2 \\&= \sum_{i=1}^p (\text{Var}(\hat{\theta}_i) + \text{Bias}^2(\hat{\theta}_i)).\end{aligned}$$



MSE of the Ridge Estimator

- To determine the mean square error for the ridge estimator, first calculate the **bias**:

$$\begin{aligned}\mathbb{E}(\hat{\beta}_{\text{RDG}}) - \beta &= (X^{\top}X + \lambda I)^{-1}(X^{\top}X + \lambda I - \lambda I)\beta - \beta \\ &= -\lambda(X^{\top}X + \lambda I)^{-1}\beta.\end{aligned}$$

- The **sum of squared biases** is thus

$$\|\mathbb{E}(\hat{\beta}_{\text{RDG}}) - \beta\|_2^2 = \lambda^2 \beta^{\top} (X^{\top}X + \lambda I)^{-2} \beta.$$

MSE of the Ridge Estimator

- For the **variance part**, write $X^T X$ in spectral form as CDC^T , where $D = \text{Diag}(\lambda)$, $\lambda = (\lambda_1, \dots, \lambda_p)$ is the vector of eigenvalues of $X^T X$, and C is an orthogonal matrix. Then

$$\begin{aligned}\text{Cov}(\hat{\beta}_{\text{RDG}}) &= \sigma^2(CDC^T + \lambda CC^T)^{-1} CDC^T (CDC^T + \lambda CC^T)^{-1} \\ &= \sigma^2 C(D + \lambda I)^{-1} D(D + \lambda I)^{-1} C^T \\ &= \sigma^2 C \begin{bmatrix} \frac{\lambda_1}{(\lambda_1 + \lambda)^2} & & \\ & \ddots & \\ & & \frac{\lambda_p}{(\lambda_p + \lambda)^2} \end{bmatrix} C^T.\end{aligned}$$

MSE of the Ridge Estimator

The mean square error of the ridge estimator $\hat{\beta}_{\text{RDG}}$ is

$$\begin{aligned}\text{mse}(\hat{\beta}_{\text{RDG}}) &= \text{tr}(\text{Cov}(\hat{\beta}_{\text{RDG}})) + \|\mathbb{E}(\hat{\beta}_{\text{RDG}}) - \beta\|_2^2 \\ &= \sigma^2 \sum_{i=1}^p \frac{\lambda_i}{(\lambda_i + \lambda)^2} + \lambda^2 \beta^\top (X^\top X + \lambda I)^{-2} \beta. \quad (6)\end{aligned}$$

- ▶ A tradeoff between bias and variance.
- ▶ Because (2) includes least squares at $\lambda = 0$, the minimizer of (6) must be in one of two cases:
 - ▶ $\lambda = 0$ (**OLS**): the least squares estimator minimizes (6).
 - ▶ $\lambda > 0$ (**ridge**): there exists some $\lambda > 0$ such that $\text{mse}(\hat{\beta}_{\text{RDG}}) < \text{mse}(\hat{\beta}_{\text{OLS}})$.

Comparison of Ridge and Least Squares Estimators

- ▶ For simplicity taking $\text{rank}(X) = p$, the ordinary least squares estimator is $\hat{\beta}_{\text{OLS}} = (X^\top X)^{-1} X^\top \mathbf{y}$. This unbiased estimator has a mean square error

$$\text{mse}(\hat{\beta}_{\text{OLS}}) = \text{tr}(\sigma^2 (X^\top X)^{-1}) = \sigma^2 \sum_{i=1}^p \frac{1}{\lambda_i}.$$

- ▶ The value of $\lambda > 0$ used for the ridge estimator stabilizes this variance term, with the greatest impact where the eigenvalues are smallest.

Shrinkage in Ridge Estimation

- ▶ Turning to the shrinkage achieved by ridge estimation, consider

$$\begin{aligned}\hat{\beta}_{\text{RDG}} &= (X^T X + \lambda I)^{-1} (X^T X) (X^T X)^{-1} X^T \mathbf{y} \\ &= (X^T X + \lambda I)^{-1} (X^T X) \hat{\beta}_{\text{OLS}} \\ &= C(D + \lambda I)^{-1} C^T C D C^T \hat{\beta}_{\text{OLS}} \\ &= C(D + \lambda I)^{-1} D C^T \hat{\beta}_{\text{OLS}} \\ &= C A C^T \hat{\beta}_{\text{OLS}}\end{aligned}$$

where

$$A = \begin{bmatrix} \frac{\lambda_1}{(\lambda_1 + \lambda)} & & \\ & \ddots & \\ & & \frac{\lambda_p}{(\lambda_p + \lambda)} \end{bmatrix}.$$

Shrinkage in Ridge Estimation

► Now calculate

$$\begin{aligned}\hat{\beta}_{\text{RDG}}^{\top} \hat{\beta}_{\text{RDG}} &= \hat{\beta}_{\text{OLS}}^{\top} \mathbf{C} \mathbf{A}^2 \mathbf{C}^{\top} \hat{\beta}_{\text{OLS}} \\ &< \hat{\beta}_{\text{OLS}}^{\top} \mathbf{C} \mathbf{A}^2 \mathbf{C}^{\top} \hat{\beta}_{\text{OLS}} + \hat{\beta}_{\text{OLS}}^{\top} \mathbf{C} (\mathbf{I} - \mathbf{A}^2) \mathbf{C}^{\top} \hat{\beta}_{\text{OLS}} \\ &= \hat{\beta}_{\text{OLS}}^{\top} \hat{\beta}_{\text{OLS}}\end{aligned}$$

showing that the L_2 norm of $\hat{\beta}_{\text{RDG}}$ is smaller than that of $\hat{\beta}_{\text{OLS}}$.

Geometric Perspective on Ridge Estimation

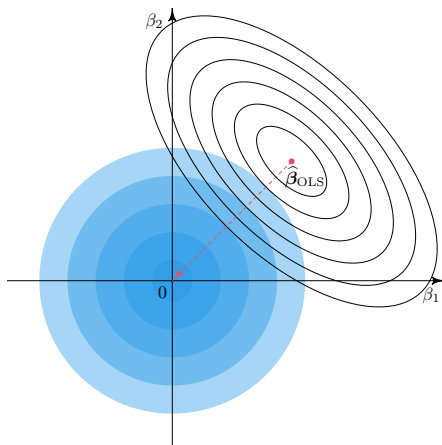


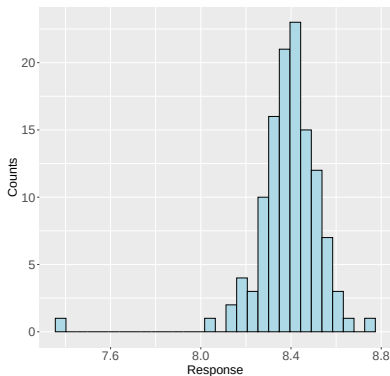
Figure: Illustration of the ridge estimator in two dimensions.

As the tuning parameter λ increases from zero, $\beta = (\beta_1, \beta_2)^\top$ moves toward the origin, as indicated by the dotted line. Each blue circle is, for some $c > 0$, a constrained region $\|\beta\|_2^2 \leq c$ of (3). The ellipses correspond to values of β producing the same value of $\text{SSE} = \|\mathbf{y} - X\beta\|_2^2$.

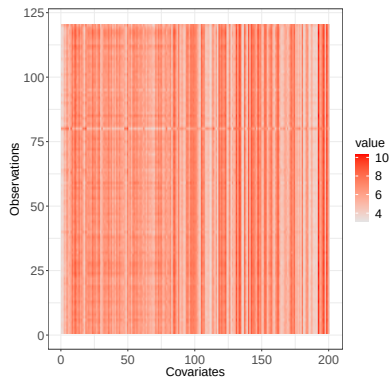
Selection of λ with Cross Validation

- ▶ The **Eye data** contains gene expression data for 200 gene probes in each of 120 tissue samples.
 - ▶ This is a **high-dimensional** modeling problem, with $p = 200$ covariates but only $n = 120$ observations.
- ▶ The **response** is the expression level of the TRIM32 gene.
- ▶ The **objective of this study** is to predict the expression level of TRIM32 from the expression levels of the 200 other genes measured in the experiment.

Example Data Visualization



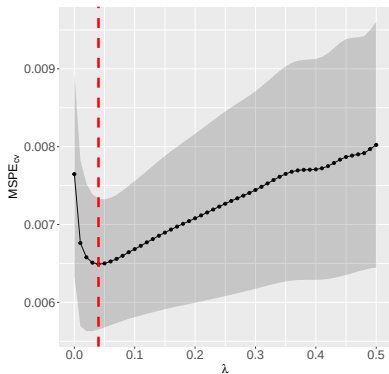
(a) Response



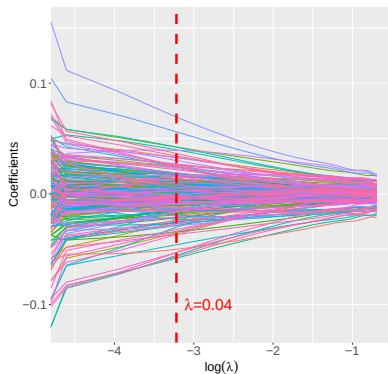
(b) Covariates

Figure: Histogram for the response in the Eye data (a) and heatmap (120×200) for the covariates (b). This is for the original (uncentered and unscaled) data. The heat map displays values of each covariate for each value of the response.

Identify the Optimal Ridge Parameter Value



(a) CV Plot



(b) Ridge trace plot

Figure: Plot of the cross-validation results used for identifying the optimal ridge parameter value ($\lambda = 0.04$)(a), and the plot of the trace of the ridge regression coefficients (b) for the Eye data. The red line shows the regression coefficients when $\lambda = 0.04$.

Bayesian Idea

- ▶ Consider the standard linear model with design matrix $X_{n \times p}$ and response vector $\mathbf{y}$:

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\epsilon} \quad (7)$$

where $\mathbb{E}(\boldsymbol{\epsilon}) = \mathbf{0}$, $\text{Cov}(\boldsymbol{\epsilon}) = \sigma^2 I$, and further assume that $\boldsymbol{\epsilon} \sim N(\mathbf{0}, \sigma^2 I)$. Here, we suppose that σ^2 is **known**.

- ▶ Set a $N_p(\mathbf{0}, \tau^2 I)$ prior on $\boldsymbol{\beta}$.

Bayesian Idea

- ▶ Then the log of the joint likelihood for β and the data is

$$\begin{aligned} l(\beta) = & -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} (\mathbf{y} - X\beta)^\top (\mathbf{y} - X\beta) \\ & - \frac{p}{2} \log(2\pi\tau^2) - \frac{1}{2\tau^2} \beta^\top \beta. \end{aligned} \quad (8)$$

- ▶ The Bayesian MAP estimator for β is found by maximizing (8) with respect to β .
- ▶ This is equivalent to minimizing

$$\ell(\beta, \lambda) = \frac{1}{2} \|\mathbf{y} - X\beta\|_2^2 + \frac{\lambda}{2} \|\beta\|_2^2,$$

where $\lambda = \sigma^2/\tau^2 > 0$.

Bayesian Idea

- ▶ If σ^2 is not known then also imposing a prior $f(\sigma^2)$ on σ^2 introduces the additional term $\log(f(\sigma^2))$ in (8).
- ▶ For instance, with the improper prior $f(\sigma^2) = 1/\sigma^2$, Bayesian MAP estimation now minimizes

$$\ell(\beta, \sigma^2, \lambda) = \frac{1}{2} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \frac{\lambda}{2} \|\beta\|_2^2 + \frac{n}{2} \sigma^2 \log(2\pi\sigma^2) + \sigma^2 \log(\sigma^2)$$

with respect to β and σ^2 .

STAT 5124: Linear Models Theory

Chapter 8: Regularization

Yili Hong

Section 8.3: Lasso

Outline

Setup and Motivation

Theoretical Foundations of Lasso

Computing of LASSO Estimates

The Bayesian Connection

Recap: ridge regression

- ▶ Recall our setup: given a response vector $\mathbf{y} \in \mathbb{R}^n$, and matrix $X_{n \times p}$ of predictor variables. Both $\mathbf{y}$ and X are centered and scaled.
- ▶ The **ridge regression estimator** of β minimizes the loss function

$$\ell(\beta, \lambda) = \frac{1}{2} \|\mathbf{y} - X\beta\|_2^2 + \frac{\lambda}{2} \|\beta\|_2^2 \quad (1)$$

with respect to β , where the tuning parameter λ is some nonnegative constant.

Motivation

- ▶ But ridge regression will never sets coefficients to zero exactly, and therefore cannot perform variable selection in the linear model.
- ▶ Ridge regression is not desirable for the purposes of **interpretation**, especially if the number of variables p is large.
- ▶ Goal: regularize least squares using an penalty to achieve **shrinkage** + **variable selection**.
- ▶ **Lasso**: Least Absolute Shrinkage and Selection Operator.

Lasso

- ▶ The L_1 penalty is

$$\lambda \|\beta\|_1 = \lambda \sum_{j=1}^p |\beta_j|$$

where $\beta = (\beta_1, \dots, \beta_p)^\top$.

- ▶ Then the penalized objective function is

$$\ell(\beta, \lambda) = \frac{1}{2} \|\mathbf{y} - X\beta\|_2^2 + \lambda \|\beta\|_1. \quad (2)$$

The LASSO estimate of $\hat{\beta}$ is found by minimizing (2) with respect to β .

Loss Function: Another Perspective

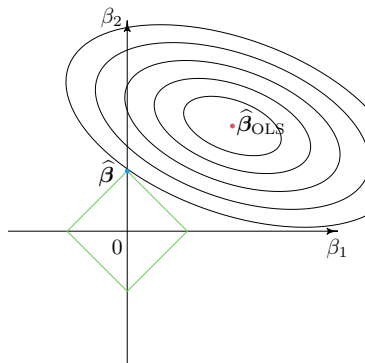
- ▶ Minimizing the penalized least squares $\ell(\beta, \lambda)$ also solves this constrained least squares problem:

$$\min_{\beta} \frac{1}{2} \|\mathbf{y} - X\beta\|_2^2 \quad \text{s. t.} \quad \|\beta\|_1 \leq c \quad (3)$$

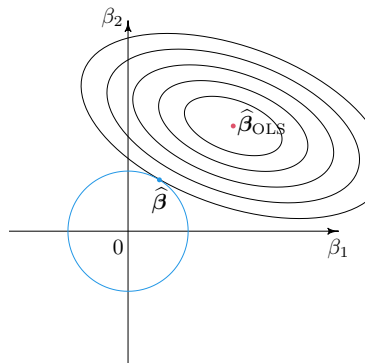
for some $c > 0$. Smaller values of c correspond to larger values of λ .

- ▶ LASSO = least squares with an L_1 -norm constraint $\Rightarrow$ shrinks β toward 0

Graphical intuition



(a) L_1 Penalty



(b) L_2 Penalty

Figure: Illustration of L_1 and L_2 penalty function.

LASSO estimator

- ▶ Unlike for ridge regression, there is usually no closed-formed expression for the LASSO estimator $\hat{\beta}$.
- ▶ However, with a special condition on the design matrix X , minimization of the penalized objective function (2) can be achieved analytically.
- ▶ With an **orthogonal, standardized** design, the loss function $\ell(\beta, \lambda)$ separates into p one-dimensional problems.

LASSO estimator

A linear model has an **orthogonal design matrix** if $X_{n \times p}$ has orthogonal columns, that is, if $X^\top X$ is a diagonal matrix. If the columns of X are standardized to have unit length, then this is $X^\top X = I_p$.

LASSO estimator

Let $X_{n \times p} = (\mathbf{x}_1, \dots, \mathbf{x}_p)$ be an **orthogonal design matrix** with standardized columns. Then the solution to the LASSO objective function (2) is $\hat{\boldsymbol{\beta}} = (\hat{\beta}_1, \dots, \hat{\beta}_p)^\top$, where $\hat{\beta}_j = S(\mathbf{x}_j^\top \mathbf{y}; \lambda)$. Here, $S(x; \gamma)$ is the **soft-thresholding function**, defined as

$$S(x; \gamma) = \begin{cases} x - \gamma, & x > \gamma \\ 0, & |x| \leq \gamma \\ x + \gamma, & x < -\gamma \end{cases} \quad (4)$$

where $\gamma > 0$ is a given constant.

LASSO estimator

With $\mathbf{X}^\top \mathbf{X} = \mathbf{I}$, the penalized objective function in (2) simplifies to

$$\begin{aligned}\ell(\boldsymbol{\beta}, \lambda) &= \sum_{j=1}^p \left(\frac{1}{2} \beta_j^2 - \mathbf{x}_j^\top \mathbf{y} \beta_j + \lambda |\beta_j| \right) + \frac{1}{2} \mathbf{y}^\top \mathbf{y} \\ &= \sum_{j=1}^p \ell(\beta_j) + \frac{1}{2} \mathbf{y}^\top \mathbf{y}\end{aligned}$$

where

$$\ell(\beta_j) = \frac{1}{2} \beta_j^2 - \mathbf{x}_j^\top \mathbf{y} \beta_j + \lambda |\beta_j|. \quad (5)$$

The minimization problem for $\ell(\boldsymbol{\beta}, \lambda)$ is thus **separable** in β_j : the problem is solved by separately minimizing each $\ell(\beta_j)$.

LASSO estimator

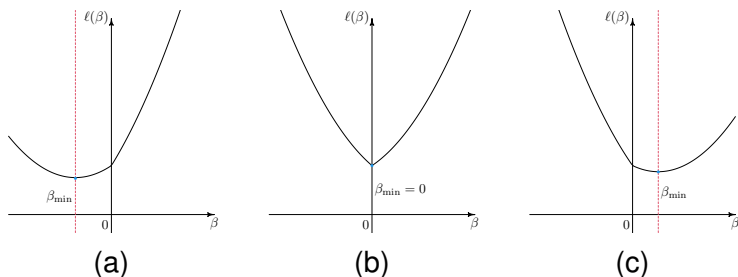


Figure: Three possible shapes of the LASSO objective function $\ell(\beta)$ for a one-dimensional β .

With a bit of algebra (**your exercise**), the solution can be written as $\hat{\beta}_j = S(\mathbf{x}_j^\top \mathbf{y}; \lambda)$.

Soft-thresholding function

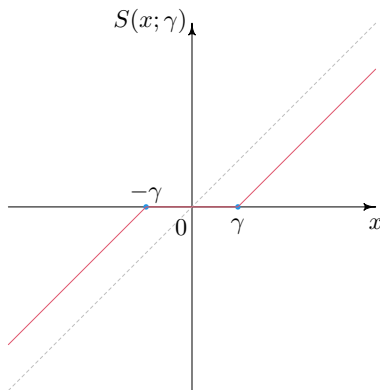


Figure: Illustration of the soft-thresholding function, showing by the solid red line.

Soft-thresholding function: key takeaway

- ▶ Under an **orthogonal, standardized** design:

$$\hat{\beta}_{\text{OLS},j} = \mathbf{x}_j^\top \mathbf{y} \quad \Rightarrow \quad \hat{\beta}_j = \mathcal{S}(\hat{\beta}_{\text{OLS},j}; \lambda).$$

- ▶ The soft-thresholding function $\mathcal{S}(x; \gamma)$ obtained in this special case is a guiding piece for the general LASSO problem.
- ▶ What $\mathcal{S}(x; \gamma)$ does:
 - sets small coefficients to zero $\Rightarrow$ **variable selection**;
 - shrinks nonzero coefficients toward 0 by an amount $\propto \lambda \Rightarrow$ **shrinkage**.

Recap: Coordinate Descent (CD) Algorithm

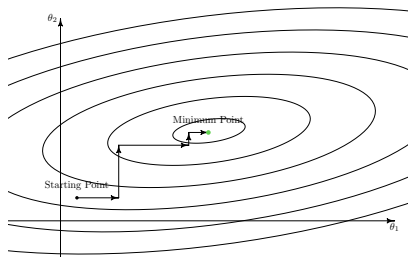


Figure: An illustration of the coordinate descent algorithm.

- ▶ The idea of CD: at each step, the objective function $\ell(\theta) = \ell(\theta_1, \theta_2, \dots, \theta_p)$ is optimized one coordinate at a time, keeping the other coordinates fixed.
- ▶ This cycle of p coordinate-wise updates is repeated until convergence.

Coordinate Descent for LASSO

- ▶ The CD minimizes (2) by iteratively updating β_j for $j = 1, \dots, p$, keeping the other $p - 1$ coefficients fixed.
- ▶ To update β_j , given the current estimate $\hat{\beta}_1, \dots, \hat{\beta}_{j-1}, \hat{\beta}_{j+1}, \dots, \hat{\beta}_p$, the objective function is

$$\begin{aligned}\ell(\beta_j) &= \frac{1}{2}(\mathbf{y} - \sum_{k \neq j} \mathbf{x}_k \hat{\beta}_k - \mathbf{x}_j \beta_j)^\top (\mathbf{y} - \sum_{k \neq j} \mathbf{x}_k \hat{\beta}_k - \mathbf{x}_j \beta_j) + \lambda |\beta_j| + c_1 \\ &= \frac{1}{2} \beta_j^2 - \mathbf{x}_j^\top (\mathbf{y} - \sum_{k \neq j} \mathbf{x}_k \hat{\beta}_k) \beta_j + \lambda |\beta_j| + c_2,\end{aligned}$$

where c_1 and c_2 are constants not involving β_j .

- ▶ Here, $\mathbf{y} - \sum_{k \neq j} \mathbf{x}_k \hat{\beta}_k$ serves the role of $\mathbf{y}$ in (5), so the solution takes the same form using soft-thresholding function:

$$\hat{\beta}_j = \mathcal{S}(\mathbf{x}_j^\top (\mathbf{y} - \sum_{k \neq j} \mathbf{x}_k \hat{\beta}_k), \lambda).$$

Coordinate Descent for LASSO

- ▶ The inputs to the soft-thresholding function can be written

$$\mathbf{x}_j^\top (\mathbf{y} - \sum_{k \neq j} \mathbf{x}_k \hat{\beta}_k) = \mathbf{x}_j^\top \mathbf{y} - \sum_{k \neq j} \mathbf{x}_j^\top \mathbf{x}_k \hat{\beta}_k.$$

- ▶ $\mathbf{x}_j^\top \mathbf{y}$ for $j = 1, \dots, p$, and $\mathbf{x}_j^\top \mathbf{x}_k$ for $j = 1, \dots, p$ and $k = 1, \dots, p$, only need to be calculated once at the data processing stage.
- ▶ The amount of computing needed in the CD updates is thus minimal – **highly efficient algorithm**.

Coordinate Descent for LASSO

- ▶ The **Auto MPG** data in Example 5.3:
 - ▶ $n = 398$, $p = 7$.
 - ▶ **Response variables**: vehicle mpg (miles per gallon) – continuous.
 - ▶ **Predictor variables**: two discrete (cylinders, model year), four continuous (displacement, horsepower, weight, acceleration), and one categorical predictor (origin).
- ▶ **Goal**: build an effective model for the vehicle mpg, in terms of 7 potential predictor variables.
- ▶ **Note**: For the categorical predictor “origin” two levels are employed, with baseline value 0 for American cars, and value 1 for European and Japanese cars.

Coordinate Descent for LASSO

A LASSO fit of a linear model can be implemented with the CD algorithm using the R package `glmnet`.

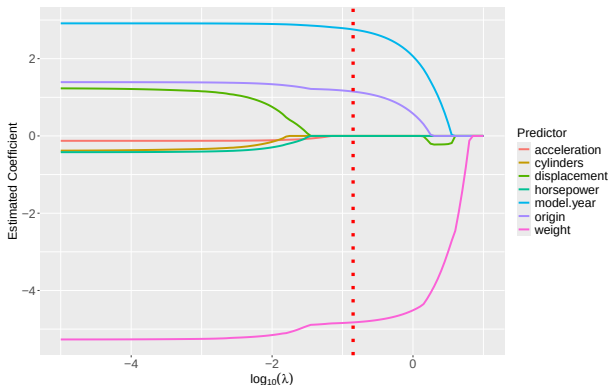
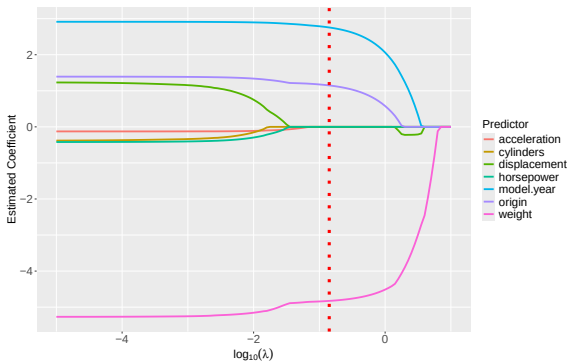


Figure: Illustration of the estimate from the LASSO regression based on the Auto MPG data.

Coordinate Descent for LASSO



- ▶ As λ increases, coefficient magnitudes shrink and many become exactly zero – **sparsity**.
- ▶ For sufficiently large λ , all coefficients are reduced to 0.
- ▶ The vertical dotted line marks $\lambda = 0.14$, which yields the smallest 5-fold MSPE_{cv} .
- ▶ At $\lambda = 0.14$, LASSO selects **year**, **origin**, and **weight**.

ADMM algorithm for LASSO

- Constrained optimization problem attacked with ADMM:

$$\begin{aligned} \min \quad & f(\mathbf{x}) + g(\mathbf{z}) \\ \text{s. t.} \quad & A\mathbf{x} + B\mathbf{z} = \mathbf{c} \end{aligned} \tag{6}$$

- (i) where $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{z} \in \mathbb{R}^m$, A is $p \times n$, B is $p \times m$, $\mathbf{c} \in \mathbb{R}^p$,
- (ii) $f(\cdot)$ and $g(\cdot)$ are **convex**, though not necessarily smooth, functions.

- The LASSO loss function can be expressed as

$$\begin{aligned} \min \quad & \frac{1}{2} \|\mathbf{y} - X\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_1 \\ \text{s. t.} \quad & I\boldsymbol{\beta} - I\boldsymbol{\theta} = \mathbf{0}. \end{aligned}$$

where $\mathbf{x} = \boldsymbol{\beta}$, $\mathbf{z} = \boldsymbol{\theta}$, $f(\mathbf{x}) = f(\boldsymbol{\beta}) = \frac{1}{2} \|\mathbf{y} - X\boldsymbol{\beta}\|_2^2$,
 $g(\mathbf{z}) = g(\boldsymbol{\theta}) = \lambda \|\boldsymbol{\theta}\|_1$, $A = I$, $B = -I$, and $\mathbf{c} = \mathbf{0}$.

ADMM algorithm for LASSO

The ADMM loss function is

$$\ell_{\rho}(\boldsymbol{\beta}, \boldsymbol{\theta}, \boldsymbol{u}) = \frac{1}{2} \|\boldsymbol{y} - X\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_1 + \frac{\rho}{2} \|\boldsymbol{\beta} - \boldsymbol{\theta} + \boldsymbol{u}\|_2^2 - \frac{\rho}{2} \|\boldsymbol{u}\|_2^2. \quad (7)$$

- ▶ $\boldsymbol{\beta}$ -step: quadratic in $\boldsymbol{\beta} \Rightarrow$ ridge-type closed form.
- ▶ $\boldsymbol{\theta}$ -step: L_1 + quadratic $\Rightarrow$ **soft-threshold**.
- ▶ $\boldsymbol{u}$ -step: dual ascent to enforce $\boldsymbol{\beta} = \boldsymbol{\theta}$.

ADMM for LASSO – key steps

► β -step

- With θ_k and $\mathbf{u}_k$ fixed, the loss function (7) can be written

$$\ell_\rho(\beta) = \frac{1}{2} \|\mathbf{y} - X\beta\|_2^2 + \frac{\rho}{2} \|\beta - \mathbf{c}\|_2^2 + d$$

where $\mathbf{c} = \theta_k - \mathbf{u}_k$ and $d = \lambda \|\theta_k\|_1 - \frac{\rho}{2} \|\mathbf{u}_k\|_2^2$ are fixed.

- Solving $\frac{\partial}{\partial \beta} \ell_\rho(\beta) = \mathbf{0}$, we have the closed-form update

$$\beta_{k+1} = (X^\top X + \rho I)^{-1} (X^\top \mathbf{y} + \rho(\theta_k - \mathbf{u}_k)).$$

► θ -step

- With β_k and $\mathbf{u}_k$ fixed, the loss function (7) can be written:

$$\ell_\rho(\theta) = \lambda \|\theta\|_1 + \frac{\rho}{2} \|\theta - (\beta_{k+1} + \mathbf{u}_k)\|_2^2$$

- The j th element of θ_{k+1} is

$$\theta_{k+1,j} = S(\beta_{k+1,j} + u_{k,j}; \frac{\lambda}{\rho}).$$

ADMM for LASSO – pseudocode

1. Fix $\rho > 0$. Initialize θ_0 and $\mathbf{u}_0$.
2. For $k = 0, 1, 2, \dots$:

2.1 **β -update (given $\theta_k, \mathbf{u}_k$).**

$$\beta_{k+1} = (X^\top X + \rho I)^{-1} (X^\top \mathbf{y} + \rho(\theta_k - \mathbf{u}_k)).$$

2.2 **θ -update (given $\beta_{k+1}, \mathbf{u}_k$).**

$$\theta_{k+1,j} = S(\beta_{k+1,j} + u_{k,j}, \frac{\lambda}{\rho}).$$

2.3 **$\mathbf{u}$ -update (given $\beta_{k+1}, \theta_{k+1}$).**

$$\mathbf{u}_{k+1} = \mathbf{u}_k + \beta_{k+1} - \theta_{k+1}.$$

3. Step 2 are repeated until convergence.

ADMM for LASSO – efficiency, scalability, and software

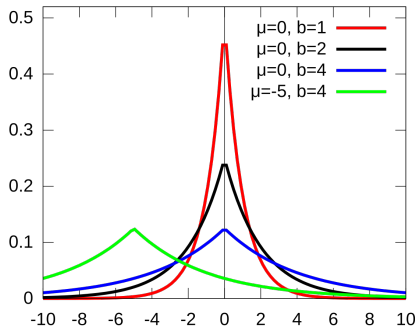
- ▶ Useful for large-scale/big-data settings.
 - ✓ With fixed ρ , precompute $(X^\top X + \rho I)^{-1}$ and $X^\top \mathbf{y}$ once (data-processing stage).
 - ✓ Precomputation + closed-form updates $\Rightarrow$ **high computational efficiency**.
 - ✓ ADMM is distribution-friendly (e.g., sum $X_i^\top X_i$, $X_i^\top \mathbf{y}_i$ across nodes).
- ▶ Easy to implement in R (package `ADMM`)..
- ▶ High-dimensional LASSO ($p > n$): the Eye data is an example.

The Bayesian Connection

- ▶ Ridge recap: $\beta \sim \mathcal{N}(0, \tau^2 I)$
 $\Rightarrow L_2$ penalty.
- ▶ LASSO: independent zero-mean **Laplace** priors
 $\beta_j \sim \text{Laplace}(\lambda)$ with density

$$f(\beta_j) = \frac{\lambda}{2} \exp(-\lambda |\beta_j|), \quad \lambda > 0.$$

- ▶ Laplace prior $\Rightarrow L_1$ penalty in the MAP objective.



MAP with Known σ^2

- ▶ With the Laplace prior, assuming σ^2 is known, the log of the joint likelihood for $\mathbf{y}$ and β is

$$\begin{aligned} l(\beta) = & -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} (\mathbf{y} - X\beta)^\top (\mathbf{y} - X\beta) \\ & - p \log\left(\frac{\lambda}{2}\right) - \lambda \sum_{j=1}^p |\beta_j|. \end{aligned} \quad (8)$$

- ▶ Maximizing (8) to obtain the Bayesian MAP estimator of β is equivalent to minimizing

$$\ell(\beta, \lambda^*) = \frac{1}{2} \|\mathbf{y} - X\beta\|_2^2 + \lambda^* \|\beta\|_1 \quad (9)$$

where $\lambda^* = \sigma^2 \lambda$. This is LASSO estimation of β .

MAP with Unknown σ^2

- ▶ Place a prior $f(\sigma^2)$ and maximize the joint posterior (MAP in β, σ^2).
- ▶ For instance, with the improper prior $f(\sigma^2) = 1/\sigma^2$, the resulting objective (up to constants) is

$$\begin{aligned}\ell(\beta, \sigma^2, \lambda) = & \frac{1}{2} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \sigma^2 \|\beta\|_1 + \sigma^2 \left(\frac{n}{2} \log(2\pi\sigma^2) \right. \\ & \left. + p \log\left(\frac{\lambda}{2}\right) - \log(f(\sigma^2)) \right)\end{aligned}$$

- ▶ It still contains a LASSO term in β , but the link between the prior scale λ and the LASSO tuning is no longer the simple $\lambda^* = \sigma^2 \lambda$.

Remarks

- ▶ The Bayesian MAP regularization equivalence explains **why** LASSO encourages sparsity (Laplace prior).
- ▶ Larger prior scale λ (or λ^*) $\Rightarrow$ stronger shrinkage, more zeros.
- ▶ Laplace is also called the **double exponential** distribution.
- ▶ **Note:** Bayesian vs frequentist differ in **post-fit inference**: quantify uncertainty, make intervals, test hypotheses, etc.

STAT 5124: Linear Models Theory

Chapter 10: Predictive Models

Yili Hong

Section 10.8: Deep Learning

Outline

Perspective

Neural Networks and Deep Learning

Activation Functions

Model Learning

Architectures

Summary and Outlook

Deep Learning in Context

- ▶ Many earlier methods can be viewed as building functions of inputs to model the response:
 - ▶ Polynomial regression, splines, local regression
 - ▶ Principal components, kernel methods
 - ▶ Tree-based models
- ▶ These methods usually end with a (generalized) linear model in the transformed features.
- ▶ Deep learning continues this idea but:
 - ▶ Uses nonlinear, non-polynomial transformations
 - ▶ Stacks many layers of transformations
 - ▶ Learns a very flexible function $f(x; w)$

Single Neuron (Unit)

- ▶ Start with p input features:

$$x_1, x_2, \dots, x_p$$

- ▶ A single neuron creates a new feature

$$z_l = \phi \left(\sum_{j=1}^p x_j \tau_{lj} + \gamma_l \right)$$

- ▶ Ingredients:
 - ▶ Weights $\tau_{l1}, \dots, \tau_{lp}$
 - ▶ Shift parameter γ_l
 - ▶ Nonlinear activation function $\phi(\cdot)$
- ▶ Interpretation: linear combination $\rightarrow$ nonlinear transformation.

Illustration

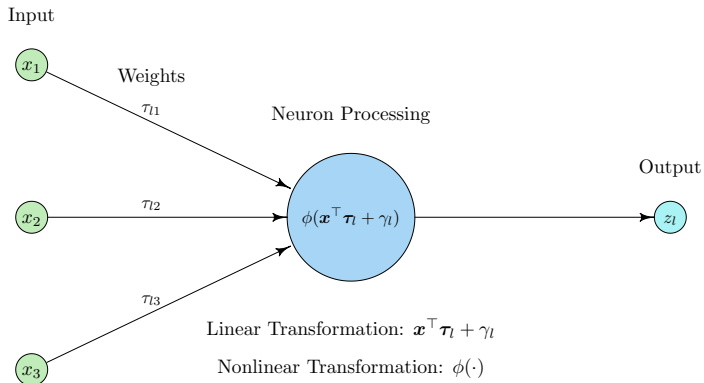


Figure: Illustration of a neuron.

Hidden Layer and Basic Architecture

- ▶ Apply the neuron construction m times to get a **hidden layer**:

$$z_1, z_2, \dots, z_m$$

- ▶ These are new, learned features of the inputs.
- ▶ The response is modeled as

$$y = \beta_0 + \sum_{l=1}^m \beta_l z_l + \varepsilon = \beta_0 + \sum_{l=1}^m \beta_l \phi \left(\sum_{j=1}^p x_j \tau_{lj} + \gamma_l \right) + \varepsilon.$$

- ▶ The layer is hidden because we never observe the z_l directly in the data.

Illustration

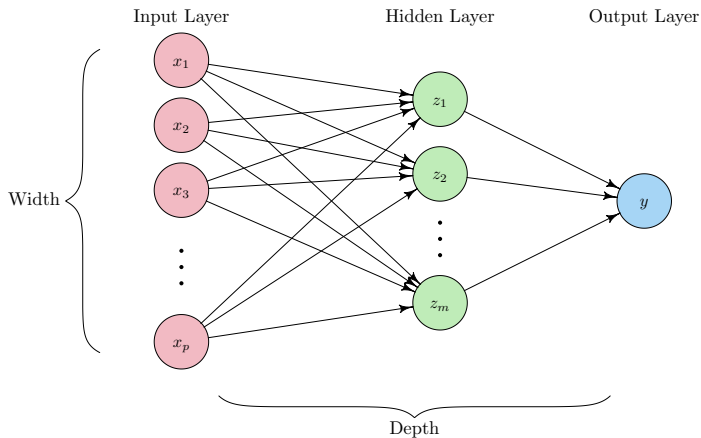


Figure: Architecture of a basic neural network.

Depth and Width

- ▶ **Width**: number of neurons in a layer (e.g., m).
- ▶ **Depth**: number of layers in the network.
- ▶ With multiple hidden layers:
 - ▶ Each hidden layer takes outputs from the previous one as inputs.
 - ▶ The network composes many nonlinear transformations.
- ▶ Terminology:
 - ▶ A network with one hidden layer: “shallow” neural net.
 - ▶ Several hidden layers: **deep** neural net (deep learning).

Multiple Hidden Layers

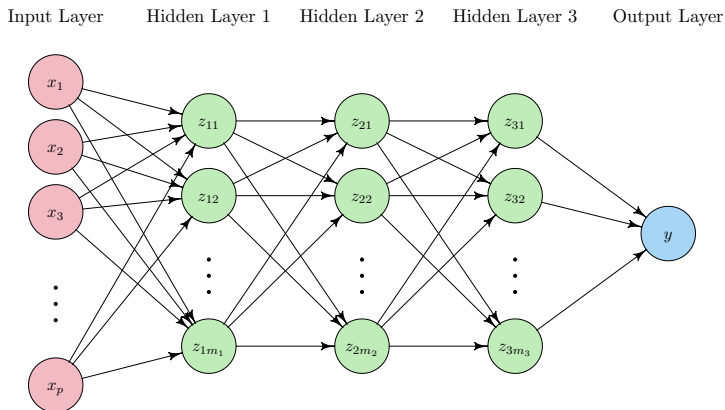


Figure: The architecture of a deep feedforward neural network, here illustrated with three hidden layers.

Matrix Formulation (Single Hidden Layer)

- Data:

$$y_i, x_{i1}, \dots, x_{ip}, \quad i = 1, \dots, n.$$

- Define matrices/vectors:

$$X = (x_{ij})_{n \times p},$$

$$\tau_l = (\tau_{l1}, \dots, \tau_{lp})^\top, \quad l = 1, \dots, m,$$

$$T = (\tau_1, \dots, \tau_m),$$

$$\gamma = (\gamma_1, \dots, \gamma_m)^\top.$$

- Hidden layer:

$$Z = \phi(XT + \mathbf{1}_n \otimes \gamma^\top).$$

- Model:

$$y = \beta_0 \mathbf{1}_n + Z\beta + \varepsilon.$$

Why Nonlinear Activation functions

- ▶ If $\phi(u) = u$ (identity), then:
 - ▶ The entire network collapses to a linear model in x .
- ▶ Nonlinear ϕ allows the network to approximate highly nonlinear functions.
- ▶ Choice of activation functions affect:
 - ▶ Smoothness of the fit
 - ▶ Gradient behavior
 - ▶ Computational convenience

Common Activation Functions

- ▶ **Sigmoid** (logistic):

$$\phi(u) = \frac{1}{1 + e^{-u}}$$

- ▶ Smooth, bounded between 0 and 1.
- ▶ Historically popular; still used in output layers for probabilities.

- ▶ **Rectified Linear Unit (ReLU):**

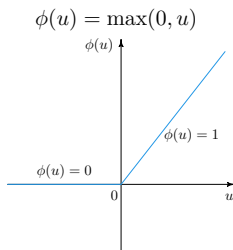
$$\phi(u) = \max(0, u)$$

- ▶ Simple, piecewise linear. Very common in modern deep nets.
- ▶ **Softplus:**

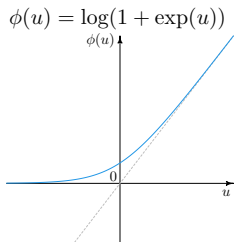
$$\phi(u) = \log(1 + e^u)$$

- ▶ Smooth approximation to ReLU.

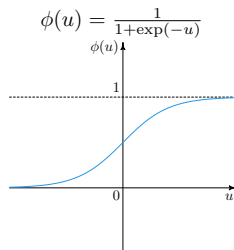
Plot of Activation Functions



(a) ReLU



(b) Softplus



(c) Sigmoid

Figure: Examples of activation functions.

Number of Parameters

- ▶ Collect all parameters into a single vector w :

$$w = (\gamma_1, \tau_1^\top, \dots, \gamma_m, \tau_m^\top, \beta_0, \beta^\top)^\top.$$

- ▶ For a deep network with:
 - ▶ p inputs ($m_0 = p$),
 - ▶ h hidden layers,
 - ▶ m_k neurons in hidden layer k ,

- ▶ One can show the number of parameters is

$$\left(\sum_{k=1}^h m_k (m_{k-1} + 1) \right) + m_h + 1.$$

- ▶ This can be very large $\Rightarrow$ risk of overfitting and identifiability issues.

Loss Functions

- ▶ Network defines $f(x; w)$ for given parameters w .
- ▶ SSE for Regression (continuous y):

$$\ell(w) = \sum_{i=1}^n (y_i - f(x_i; w))^2$$

- ▶ Cross entropy for Binary classification (probabilities $f(x_i; w)$):

$$\ell(w) = \sum_{i=1}^n \left[y_i \log f(x_i; w) + (1 - y_i) \log (1 - f(x_i; w)) \right]$$

Gradient Descent

- ▶ Optimization typically uses some form of gradient descent:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \delta \nabla \ell(\mathbf{w}^{(k)})$$

where δ is the learning rate(step size).

- ▶ Gradients involve many nested compositions due to multiple layers, which makes it hard to calculate the gradient.
- ▶ Naive differentiation would repeatedly re-compute many derivatives.
- ▶ **Backpropagation** is an efficient way to apply the chain rule:
 - ▶ Computes gradients layer by layer.
 - ▶ Reuses intermediate results instead of recomputing them.

Backpropagation: Big Picture

- **Goal:** compute the gradient of the loss

$$\nabla \ell(\mathbf{w}) = \frac{\partial \ell(\mathbf{w})}{\partial \mathbf{w}} = \sum_{i=1}^n \frac{\partial \ell_i(\mathbf{w})}{\partial \mathbf{w}}$$

for all network parameters $\mathbf{w}$ (weights and biases).

- Deep nets have many hidden layers:
 - Parameters appear in *nested* functions.
 - Example with two hidden layers:

$$\mathbf{z}_\ell^{(1)} = \phi \left(\sum_{j_1=1}^p \mathbf{x}_{j_1} \tau_{\ell j_1}^{(1)} + \gamma_\ell^{(1)} \right),$$

$$\mathbf{z}_\ell^{(2)} = \phi \left(\sum_{j_2=1}^{m_1} \tau_{\ell j_2}^{(2)} \mathbf{z}_{j_2}^{(1)} + \gamma_\ell^{(2)} \right).$$

Backpropagation: Chain Rule Idea

- ▶ For one training case, think of a computation graph:

$$x \rightarrow \text{hidden layers} \rightarrow \hat{y} = f(x; w) \rightarrow \ell$$

- ▶ **Forward pass:**

- ▶ Start from inputs x .
- ▶ Compute each layer's outputs using current parameters w .
- ▶ At the end, obtain prediction $f(x; w)$ and loss ℓ .

- ▶ **Backward pass:**

- ▶ Start from $\frac{\partial \ell}{\partial \hat{f}}$ at the output.
- ▶ Use the chain rule to move backwards through the layers:

$$\frac{\partial \ell}{\partial w^{(L)}} \Rightarrow \frac{\partial \ell}{\partial z^{(L-1)}} \Rightarrow \dots \Rightarrow \frac{\partial \ell}{\partial w^{(1)}}.$$

- ▶ At each layer:

- ▶ Compute derivatives w.r.t. that layer's weights and biases.
- ▶ Pass “error signals” back to the previous layer.

Backpropagation

- ▶ Result: all partial derivatives $\partial \ell / \partial w$ are computed with one forward pass + one backward pass.
- ▶ Key advantages of backpropagation:
 - ▶ Exploits the layered structure of the network.
 - ▶ Avoids redundant gradient computations.

Stochastic Gradient Descent (SGD)

- ▶ For large n , computing gradients on full data each step is expensive.
- ▶ **Minibatch idea:**
 - ▶ Sample a subset of indices $B \subset \{1, \dots, n\}$ at each step.
 - ▶ Approximate gradient using only $i \in B$:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \delta \sum_{i \in B} \frac{\partial \ell_i(\mathbf{w})}{\partial \mathbf{w}}.$$

- ▶ **Stochastic gradient descent:**
 - ▶ Uses random minibatches at each iteration.
 - ▶ Introduces noise in gradient estimates but is much faster.
- ▶ Modern variants:
 - ▶ Momentum, adaptive learning rates (e.g., Adam).

Tensors and Software

- ▶ Inputs (especially images, videos) are naturally represented as **tensors**.
- ▶ Linear operations in the network can be written as tensor operations.
- ▶ Popular software:
 - ▶ `PyTorch`, `TensorFlow` (Python)
 - ▶ R interfaces exist, but typically call the underlying Python libraries.
- ▶ Hardware acceleration (GPUs, TPUs) is crucial for large models.

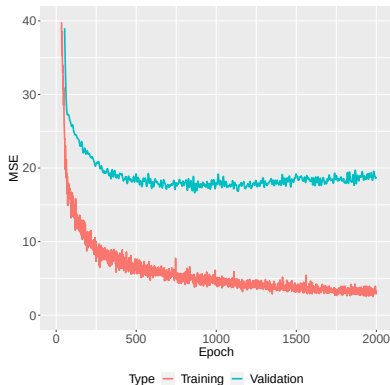
Deep Forward Neural Networks (DFNN)

- ▶ Also called fully connected or feedforward networks.
- ▶ Key property:
 - ▶ Every neuron in a layer is connected to *every* neuron in the next layer.
- ▶ Typical structure:
 - ▶ Input layer: original features.
 - ▶ Several hidden layers with nonlinear activations.
 - ▶ Output layer: appropriate for regression or classification.
- ▶ Very flexible but can have many parameters.

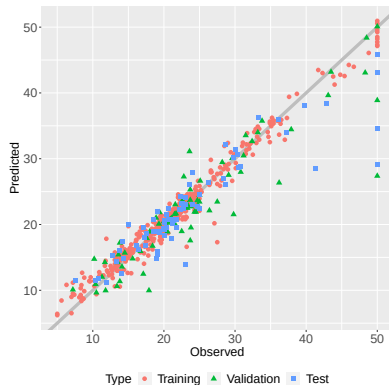
Boston Housing Price Data: Model Setup

- ▶ Boston Housing Price data (1970s): 506 observations, 13 covariates.
- ▶ Goal: predict median home value y (in thousands of dollars).
- ▶ Deep feedforward neural network (DFNN) with 3 hidden layers.
- ▶ Hidden layer sizes: $(m_1, m_2, m_3) = (64, 32, 16)$.
- ▶ ReLU activation in hidden layers; identity output for continuous y .
- ▶ Loss function: mean squared error (MSE).
- ▶ Small DL model chosen due to limited dataset size.

History



(a) Training History



(b) Observed vs. Predicted

Figure: Assessing a DL model fit with dropout for the Boston housing price data. The axes in (b) are in units of \$1,000.

Training and Results

- ▶ Covariates standardized for numerical stability.
- ▶ Dropout rate 0.2 on first two hidden layers to reduce overfitting.
- ▶ Data split: 70% training, 15% validation, 15% test.
- ▶ Optimization: Adam algorithm; trained for 2000 epochs.
- ▶ Validation MSE suggests overfitting begins around epoch 800.
- ▶ Final MSE: training 2.544, validation 16.631, test 16.478.
- ▶ Test RMSE = 4.059 (\$4,059); high-priced homes harder to predict.

Convolutional Neural Networks (CNNs)

- ▶ DFNNs connect every neuron to the next layer:
 - ▶ Many parameters, no explicit spatial structure.
- ▶ Typical components:
 - ▶ Input layer
 - ▶ Convolutional layers (Conv)
 - ▶ Pooling layers
 - ▶ Fully connected layers at the end
- ▶ Applications:
 - ▶ Image and video classification
 - ▶ Other structured data (e.g., time series, signals)

Example

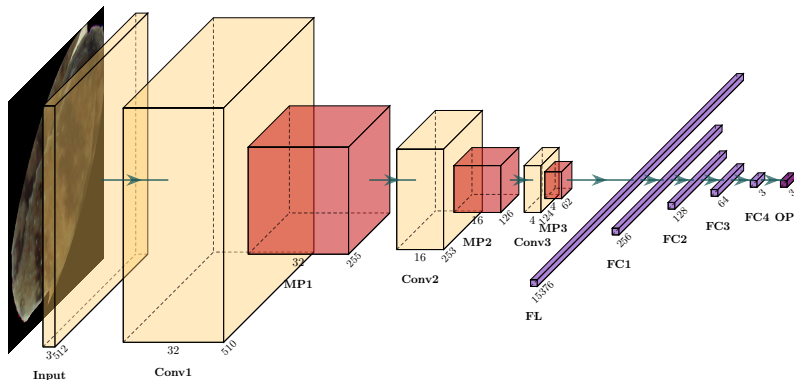


Figure: Diagram of CNN that is used for classification. Here “Conv1” means convolution layer 1, “MP1” means maximum pooling after Conv1, “FL” means flattened layer, “FC1” means fully connected layer 1, and “OP” means output layer.

Convolutional Layer

- ▶ A convolutional layer uses **filters** (kernels) to scan the input.
- ▶ Each filter:
 - ▶ Is a small weight matrix, making the dimension lower e.g., 3×3 .
 - ▶ Scans across the image (or feature map).
 - ▶ Produces a *feature map* indicating how strongly that pattern is present.
- ▶ Filters are learned from data:
 - ▶ Early layers may detect simple edges or textures.
 - ▶ Deeper layers detect more complex shapes or motifs.

Example of Convolutional Layer

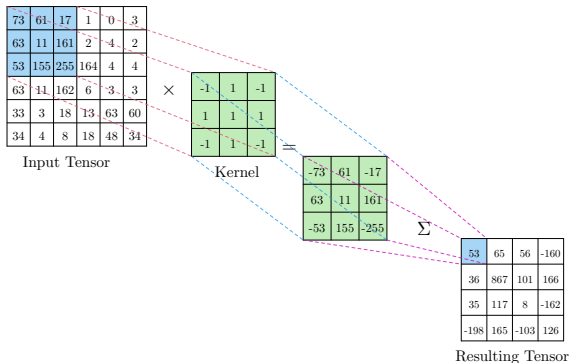


Figure: Illustration of convolution in CNN

Pooling Layer

- ▶ Pooling reduces spatial resolution of feature maps.
- ▶ **Max pooling** example:
 - ▶ Take non-overlapping windows (e.g., 3×3).
 - ▶ Replace each window by its maximum value.
- ▶ Benefits:
 - ▶ Reduces dimensionality and computation.
 - ▶ Introduces some translation invariance.
 - ▶ Helps control overfitting.

Example of Max Pooling

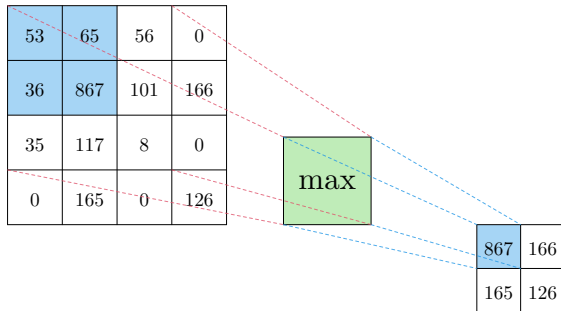


Figure: Illustration of max pooling in CNN

Flatten and Fully Connected Layers

- ▶ After several Conv + Pool blocks:
 - ▶ Output is typically a tensor ($\text{height} \times \text{width} \times \text{channels}$).
- ▶ **Flatten** layer:
 - ▶ Reshapes tensor output into a single long vector.
- ▶ Fully connected (dense) layers:
 - ▶ Similar to DFNN layers.
 - ▶ Use the flattened vector as input and produce outputs suited to the task (e.g., class scores).
- ▶ CNN architectures often repeat Conv + Pool several times, then use a small stack of dense layers.

Example: Frog Classification with CNN

- ▶ Data: photographs of individual frogs, each labeled by species (clade).
- ▶ Goal: classify each image to a clade using a CNN.
- ▶ Training:
 - ▶ 5-fold cross-validation (CV) to evaluate performance.
 - ▶ CNN with multiple Conv, pooling, flatten, and fully connected layers.

Frog Classification: Confusion Matrix

Predicted	True Clade			
	4	5	8	12
4	27	1	1	8
5	0	9	2	0
8	6	10	70	6
12	11	2	4	35

- ▶ Many images are correctly classified (diagonal entries).
- ▶ The overall accuracy of 73.1% was achieved.

Key Takeaways

- ▶ Deep learning builds complex functions by composing many simple nonlinear units.
- ▶ Neural networks:
 - ▶ Use nonlinear activations to move beyond linear models.
 - ▶ Are trained by gradient-based methods and backpropagation.
- ▶ Architectures:
 - ▶ Deep forward networks for general tabular data.
 - ▶ CNNs for images and structured inputs.
- ▶ Regularization, careful optimization, and appropriate architecture choices are crucial for good performance.

Further Directions

- ▶ Other important architectures:
 - ▶ Recurrent networks for sequences
 - ▶ Transformer models for sequence and language modeling
- ▶ Deep learning is evolving quickly:
 - ▶ New architectures and training tricks appear regularly.
 - ▶ Core ideas remain:
 - ▶ Neurons, layers, activations
 - ▶ Tensor computations
 - ▶ Gradient-based learning

STAT 5124: Linear Models Theory

Chapter 11: Nonlinear Relationship

Yili Hong

Section 11.2: Nonlinear Regression Models
Section 11.3: Nonlinear Mixed Models

Outline

Motivating Examples

Nonlinear Models with Additional Explanatory Variables

Parameter Estimation

Statistical Inference

Nonlinear Mixed Models

Basics of Nonlinear Models

- ▶ Consider an exponential decay model for biochemical oxygen demand (BOD) data. BOD was measured in samples of incubated stream water held at constant temperature for a week. Measurements were taken at $t = 1, 2, 3, 4, 5$, and 7 days.
- ▶ The model for $y = \text{BOD}$ in a sample is

$$\mathbb{E}(y) = \gamma_1(1 - \exp(-\gamma_2 t)). \quad (1)$$

- ▶ Here γ_1 is the ultimate BOD and γ_2 controls the rate of the BOD reaction.
- ▶ Equation (1) illustrates the mean function $g(t; \gamma)$, which is nonlinear in the parameters. Here $g(t; \gamma) = \gamma_1(1 - \exp(-\gamma_2 t))$ with $\gamma = (\gamma_1, \gamma_2)^\top$.

Basics of Nonlinear Models

- ▶ The rate y through time of some chemical reactions can be described by a differential equation of the form

$$\frac{dg(t)}{dt} = \frac{\alpha}{\nu \exp(\mu)} \left(\frac{g(t)}{\alpha} \right)^{1-\nu} \left(1 - \frac{g(t)}{\alpha} \right)^{1+\nu}. \quad (2)$$

- ▶ This is a form of the Prout-Tompkins equation in kinetic models. Here α , μ , and $\nu > 0$ are parameters, and $g(0) = 0$.
- ▶ One solution of this differential equation for any given $\gamma = (\alpha, \mu, \nu)^\top$ is

$$g(t; \gamma) = \frac{\alpha}{1 + \left(\frac{t}{\exp(\mu)} \right)^{-1/\nu}} = \frac{\alpha}{1 + \exp\left(-\frac{\log(t) - \mu}{\nu}\right)}. \quad (3)$$

- ▶ The resulting nonlinear model is $\mathbb{E}(y) = g(t; \gamma)$.

Basics of Nonlinear Models

Theophylline is an orally administered medication for the treatment of breathing difficulties arising from asthma and other lung conditions.

For one individual, data on bloodstream concentration of theophylline as a function of time t .

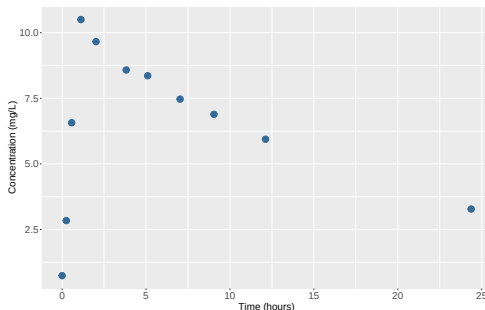


Figure: Drug concentration through time for one subject in the theophylline study.

Basics of Nonlinear Models

Based on a simple compartment model from certain differential equations, a nonlinear function for modeling theophylline concentration y , following oral dose d , is

$$\mathbb{E}(y) = g(t; \gamma) = \frac{\kappa d}{\nu(\kappa - \xi/\nu)} (\exp(-\xi t/\nu) - \exp(-\kappa t)), \quad (4)$$

where κ is the fractional rate of absorption, ξ is the clearance rate, and ν is the volume of distribution. The parameter vector is $\gamma = (\kappa, \xi, \nu)^\top$.

Basics of Nonlinear Models

- ▶ Data from nonlinear models such as those in the basic examples above can be represented as (y_i, t_i) for $i = 1, \dots, n$, where y_i is the response, t_i is the measuring point, and n is the total number of observations.
- ▶ To this setup, we now consider the introduction of additional covariates $\mathbf{x}$.

Linking Model Parameters to Explanatory Variables

- ▶ The coating data come from a degradation study of polymeric paints and coatings. The primary goal is to understand degradation over time, with temperature, UV intensity, and relative humidity also recorded.
- ▶ Here we focus on two coated units, NSS-0001-12 at 25°C and NSS-0005-12 at 35°C, both measured at 100% UV intensity and 0% relative humidity.

Linking Model Parameters to Explanatory Variables

Plots the degradation response over time (days) for these two units. Clearly, a basic model for degradation as a function of time t should include temperature as a covariate.

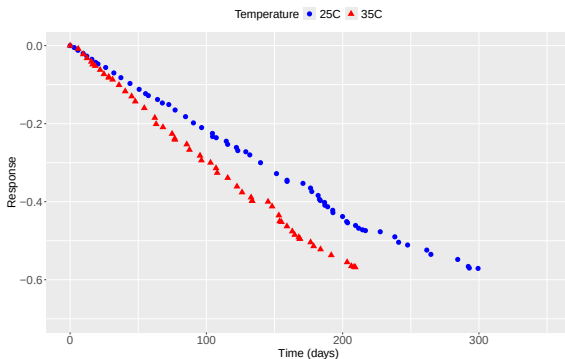


Figure: Plot of the coating data from two temperature levels.

Linking Model Parameters to Explanatory Variables

- ▶ When explanatory variables are included, the data for fitting a nonlinear model is denoted $(y_i, t_i, \mathbf{x}_i)$ for $i = 1, \dots, n$, where $\mathbf{x}$ is the vector of explanatory variables other than t .
- ▶ Understanding the effect of covariates on the response/ t relationship becomes an important component of the modeling process. This is approached by linking the covariates to parameters in the nonlinear function $g(t; \gamma)$.

Linking Model Parameters to Explanatory Variables

The BOD data includes measurements on two water samples collected from the same source on two different days in 1961. The $6 + 6 = 12$ measurements from these samples indicate two distinct curves corresponding to the two collection times.

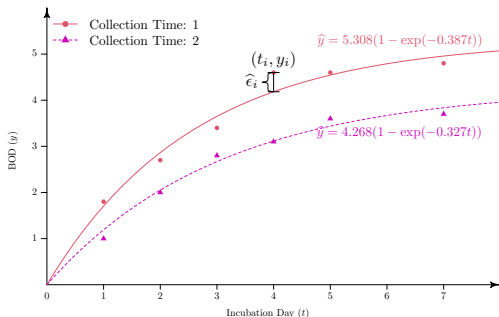


Figure: A fit of model to the BOD data.

Linking Model Parameters to Explanatory Variables

Coding collection time 1 as $x_i = 0$, and collection time 2 as $x_i = 1$, first consider incorporating this group indicator into both parameters γ_1 and γ_2 in $g(t; \gamma)$. A route for this with a linear predictor is

$$\gamma_{i1} = \exp(\mathbf{x}_i^\top \boldsymbol{\beta}_1) \quad \text{and} \quad \gamma_{i2} = \exp(\mathbf{x}_i^\top \boldsymbol{\beta}_2), \quad (5)$$

where $\mathbf{x}_i^\top = (1, x_i)$ for $i = 1, \dots, 12$ corresponds to a baseline representation for the two collection time groups.

Linking Model Parameters to Explanatory Variables

The linking (5) of γ_1 and γ_2 to $\mathbf{x}$ produces a γ_1, γ_2 pair in (1) that depends on group:

$$(\gamma_1, \gamma_2) = \begin{cases} (\exp(\beta_{10}), \exp(\beta_{20})) & \text{if } x_i = 0, \\ (\exp(\beta_{10} + \beta_{11}), \exp(\beta_{20} + \beta_{21})) & \text{if } x_i = 1. \end{cases}$$

These two pairs of two parameters are functions of the four underlying, unknown parameters $\boldsymbol{\beta}^\top = (\boldsymbol{\beta}_1^\top, \boldsymbol{\beta}_2^\top)$, with $\boldsymbol{\beta}_1 = (\beta_{10}, \beta_{11})^\top$ and $\boldsymbol{\beta}_2 = (\beta_{20}, \beta_{21})^\top$.

Linking Model Parameters to Explanatory Variables

The expressions (5) are analogous to what was done for GLMs. Inverting those transformations gives parameter pair α_i for the i th observation in the familiar linear form:

$$\alpha_i = \begin{bmatrix} \log(\gamma_{i1}) \\ \log(\gamma_{i2}) \end{bmatrix} = \begin{bmatrix} \mathbf{x}_i^\top \beta_1 \\ \mathbf{x}_i^\top \beta_2 \end{bmatrix} = \begin{bmatrix} \mathbf{x}_i^\top & \mathbf{0}^\top \\ \mathbf{0}^\top & \mathbf{x}_i^\top \end{bmatrix} \begin{bmatrix} \beta_1 \\ \beta_2 \end{bmatrix} \stackrel{\text{denote}}{=} \mathbf{X}_i \boldsymbol{\theta}. \quad (6)$$

Parameter Estimation

- ▶ For data $y_1, \dots, y_n$ we now write this general form for a nonlinear model:

$$y_i = g(t_i, \mathbf{x}_i; \boldsymbol{\theta}) + \epsilon_i, \quad i = 1, \dots, n, \quad (7)$$

where the error terms ϵ_i are independent with $\mathbb{E}(\epsilon_i) = 0$ and $\text{Var}(\epsilon_i) = \sigma^2$. The unknown parameters in (7) are the members of γ that are not linked to covariates, and the linking parameters for the other members of γ .

Parameter Estimation

- ▶ Write $\mathbf{g}(\theta)$ for the vector of the $g(t_i, \mathbf{x}_i; \theta)$:

$$\mathbf{g}(\theta) = \begin{bmatrix} g(t_1, \mathbf{x}_1; \theta) \\ \vdots \\ g(t_n, \mathbf{x}_n; \theta) \end{bmatrix}_{n \times 1}.$$

- ▶ Then the model in vector form is

$$\mathbf{y} = \mathbf{g}(\theta) + \epsilon \quad (8)$$

with $\mathbb{E}(\epsilon) = \mathbf{0}$ and $\text{Cov}(\epsilon) = \sigma^2 I_n$.

Parameter Estimation

- ▶ A fit of (8) can be achieved by minimizing an appropriate loss function. Here we proceed with least squares loss, leading to what is commonly called nonlinear least squares (NLS).
- ▶ We seek to minimize the sum of the squares of the vertical distance.

Parameter Estimation

- ▶ More formally, the goal is to solve the optimization problem $\min_{\theta} \ell(\theta)$, where

$$\ell(\theta) = (\mathbf{y} - \mathbf{g}(\theta))^{\top} (\mathbf{y} - \mathbf{g}(\theta)) = \|\mathbf{y} - \mathbf{g}(\theta)\|_2^2. \quad (9)$$

- ▶ Taking the derivative of ℓ with respect to θ and setting the result to zero gives the NLS equations:

$$\frac{\partial \ell(\theta)}{\partial \theta} = -2X_{\theta}^{\top} (\mathbf{y} - \mathbf{g}(\theta)) \stackrel{\text{set}}{=} \mathbf{0} \quad \text{with} \quad X_{\theta}^{\top} = \frac{\partial \mathbf{g}(\theta)}{\partial \theta}. \quad (10)$$

- ▶ Due to the nonlinearity, there is usually no closed-form solution to these equations. Minimizing (9) will require an algorithmic approach.

Iterative Least Squares

- ▶ At each step, a linear function from a Taylor series expansion is used to approximate the nonlinear function $g(\theta)$.

At iteration l , with current value $\hat{\theta}_l$, this is

$$\begin{aligned} g(t_i, \mathbf{x}_i; \theta) &\approx g(t_i, \mathbf{x}_i; \hat{\theta}_l) \\ &+ \left(\frac{\partial g(t_i, \mathbf{x}_i; \theta)}{\partial \theta} \Big|_{\theta=\hat{\theta}_l} \right)^\top (\theta - \hat{\theta}_l), \\ i &= 1, \dots, n. \end{aligned} \tag{11}$$

The equations (11) can be more compactly written as

$$\mathbf{g}(\theta) \approx \mathbf{g}(\hat{\theta}_l) + X_l \delta_l \tag{12}$$

where $\delta_l = (\theta - \hat{\theta}_l)$ and X_l is the $n \times p$ matrix with i th row

$$\frac{\partial g(t_i, \mathbf{x}_i; \theta)}{\partial \theta} \Big|_{\theta=\hat{\theta}_l}.$$

Iterative Least Squares

- ▶ Let $\tilde{\mathbf{y}} = \mathbf{y} - \mathbf{g}(\hat{\boldsymbol{\theta}}_l)$. Replacing $\mathbf{g}(\boldsymbol{\theta})$ in (8) with the approximation (12) gives

$$\tilde{\mathbf{y}} = \mathbf{X}_l \boldsymbol{\delta}_l + \boldsymbol{\epsilon},$$

a linear model that is fit by

$$\hat{\boldsymbol{\delta}}_l = (\mathbf{X}_l^\top \mathbf{X}_l)^{-1} \mathbf{X}_l^\top \tilde{\mathbf{y}}.$$

- ▶ The updated $\boldsymbol{\theta}$ is now

$$\hat{\boldsymbol{\theta}}_{l+1} = \hat{\boldsymbol{\theta}}_l + \hat{\boldsymbol{\delta}}_l.$$

- ▶ This algorithm stops when the change in $\hat{\boldsymbol{\theta}}$ is small.
- ▶ Analogous to least squares estimation with a linear model, an estimator of σ^2 is then

$$s^2 = \frac{(\mathbf{y} - \mathbf{g}(\hat{\boldsymbol{\theta}}))^\top (\mathbf{y} - \mathbf{g}(\hat{\boldsymbol{\theta}}))}{n - p} = \frac{\|\mathbf{y} - \mathbf{g}(\hat{\boldsymbol{\theta}})\|_2^2}{n - p}. \quad (13)$$

Statistical Inference

- ▶ With normally distributed errors $\epsilon \sim N(\mathbf{0}, \sigma^2 I)$, tests and confidence intervals for the parameters can be constructed. Writing $\psi = (\boldsymbol{\theta}^\top, \sigma^2)^\top$, the log-likelihood is

$$l(\psi) = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log(\sigma^2) - \frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{g}(\boldsymbol{\theta}))^\top (\mathbf{y} - \mathbf{g}(\boldsymbol{\theta})). \quad (14)$$

- ▶ It is easy to show that the MLE for $\boldsymbol{\theta}$ is the minimizer of (9), and the MLE for the variance σ^2 is

$$\hat{\sigma}^2 = \frac{\|\mathbf{y} - \mathbf{g}(\hat{\boldsymbol{\theta}})\|_2^2}{n} = \frac{(n-p)}{n} s^2.$$

Statistical Inference

For the nonlinear model (8) with $\epsilon \sim N(\mathbf{0}, \sigma^2 I)$, let $\hat{\psi}$ be the MLE for $\psi = (\theta^\top, \sigma^2)^\top$. Then

$$\hat{\psi} \sim N_{p+1}(\psi, \Sigma_\psi).$$

Further, a consistent estimator of the covariance matrix Σ_ψ is provided by $\hat{\Sigma}_\psi = I_{\hat{\psi}}^{-1}$, the inverse of the local information matrix:

$$I_{\hat{\psi}} = -\frac{\partial^2 l(\psi)}{\partial \psi \partial \psi^\top} \Big|_{\psi=\hat{\psi}} = \frac{n}{\hat{\sigma}^2} \begin{bmatrix} \frac{1}{2n} \cdot \frac{\partial^2 \ell(\theta)}{\partial \theta \partial \theta^\top} \Big|_{\theta=\hat{\theta}} & \mathbf{0} \\ \mathbf{0}^\top & \frac{1}{2\hat{\sigma}^2} \end{bmatrix} \quad (15)$$

where $l(\psi)$ is the log-likelihood (14) and $\ell(\theta)$ is the NLS loss function (9).

Statistical Inference

- ▶ In many software packages (e.g., the `nls` package in R), a linear approximation is used to approximate the nonlinear mean structure. This approximation (compare (12)) is

$$\mathbf{g}(\hat{\boldsymbol{\theta}}) \approx \mathbf{g}(\boldsymbol{\theta}) + X_{\boldsymbol{\theta}}(\hat{\boldsymbol{\theta}} - \boldsymbol{\theta}),$$

where $X_{\boldsymbol{\theta}}$ is defined in (10).

- ▶ Using this approximation with the model gives

$$\hat{\boldsymbol{\theta}} - \boldsymbol{\theta} \approx (X_{\boldsymbol{\theta}}^{\top} X_{\boldsymbol{\theta}})^{-1} X_{\boldsymbol{\theta}}^{\top} \boldsymbol{\epsilon} \quad \text{and} \quad (n - p)s^2 \approx \boldsymbol{\epsilon}^{\top} (I - P_{X_{\boldsymbol{\theta}}}) \boldsymbol{\epsilon}.$$

Here $P_{X_{\boldsymbol{\theta}}}$ is the projector onto the column space of $X_{\boldsymbol{\theta}}$.

Statistical Inference

Provided n is large,

- ▶ $\hat{\theta} \dot{\sim} N_p(\theta, \sigma^2(X_{\theta}^{\top} X_{\theta})^{-1})$
- ▶ $(n - p)s^2/\sigma^2 \dot{\sim} \chi^2(n - p)$
- ▶ $\hat{\theta}$ and s^2 are asymptotically independent.

Statistical Inference

The data was collected in an experiment conducted on the pharmacokinetics of theophylline.

Here we examine one subject, Subject #1 with dose $d = 4.02$, and fit model (16) to the concentration of the drug in their blood. Recall that:

$$\mathbb{E}(y) = g(t; \gamma) = \frac{\kappa d}{\nu(\kappa - \xi/\nu)} (\exp(-\xi t/\nu) - \exp(-\kappa t)), \quad (16)$$

where κ is the fractional rate of absorption, ξ is the clearance rate, and ν is the volume of distribution. The parameter vector is $\gamma = (\kappa, \xi, \nu)^\top$.

Statistical Inference

Estimates for the model parameters κ , ν , and ξ are summarized:

Table: Parameter estimates for the fitted curve to the data from one individual from the Theophylline study.

	Estimate	Std Error	<i>t</i> -value	P-value
κ	1.7774	0.3071	5.787	0.0004
ξ	0.0199	0.0025	7.857	< 0.0001
ν	0.3692	0.0222	16.605	< 0.0001

Statistical Inference

Plots the observations obtained from this subject as a function of $t = \text{time}$.

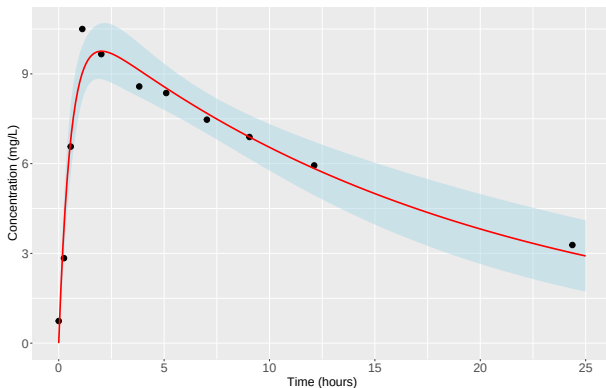


Figure: Fit of a nonlinear model to theophylline concentration over time for one individual.

Statistical Inference

Consider fitting a nonlinear model to the coating data for two units at two different temperatures.

Define a grouping covariate x with $x_i = 0$ (25°C) and $x_i = 1$ (35°C).

The vertical-shift parameter μ in (3) is linked to x by $\mu_i = \mathbf{x}_i^\top \boldsymbol{\beta}$, where $\mathbf{x}_i = (1, x_i)^\top$ and $\boldsymbol{\beta} = (\beta_0, \beta_1)^\top$.

The linear form linking the three parameters for observation i to the covariate is

$$\boldsymbol{\alpha}_i = \begin{bmatrix} \alpha_i \\ \nu_i \\ \mu_i \end{bmatrix} = \begin{bmatrix} \alpha \\ \nu \\ \mathbf{x}_i^\top \boldsymbol{\beta} \end{bmatrix} = \begin{bmatrix} 1 & 0 & \mathbf{0}^\top \\ 0 & 1 & \mathbf{0}^\top \\ 0 & 0 & \mathbf{x}_i^\top \end{bmatrix} \begin{bmatrix} \alpha \\ \nu \\ \boldsymbol{\beta} \end{bmatrix} = \mathbf{X}_i \boldsymbol{\theta}. \quad (16)$$

The parameters $\alpha_i = \alpha$ and $\nu_i = \nu$ are not linked and so do not change across observations.

Statistical Inference

Parameter estimates for the model are summarized:

Table: Parameter estimation for nonlinear regression model based on coating data with two temperature levels.

Parameter	Estimate	Std Error	<i>t</i> -value	P-value
α	-1.5515	0.1197	-12.96	< 0.001
ν	0.8411	0.0192	43.73	< 0.001
β_0	6.1148	0.1089	56.13	< 0.001
β_1	-0.3171	0.0054	-58.13	< 0.001

Statistical Inference

Plot of the resulting curves for the two temperature levels. The fit appears quite good at 35°C. The fit is less satisfactory at 25°C, which could be due either to model inadequacy or to variation associated with this particular unit.

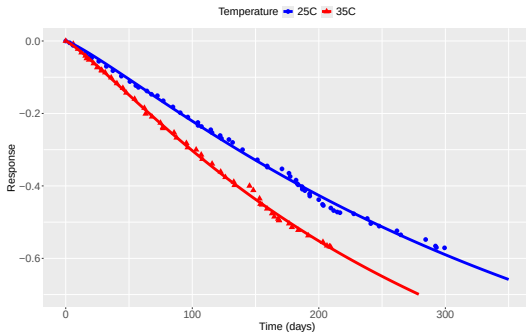


Figure: Nonlinear curve fits for two temperature levels of the coating data.

Nonlinear Mixed Models (NLMM)

- ▶ s subjects, with subject i measured at $t_{i1}, \dots, t_{in_i}$.
- ▶ Observed responses: $y_{i1}, \dots, y_{in_i}$.
- ▶ Error terms: $\epsilon_{i1}, \dots, \epsilon_{in_i}$.
- ▶ Covariates (if employed) depend on subject i , but not on time t_{ij} .

The basic NLMM is

$$y_{ij} = g(t_{ij}; \alpha_i) + \epsilon_{ij} \quad (17)$$

for $i = 1, \dots, s$ and $j = 1, \dots, n_i$. $g(t; \gamma)$ is the basic mean function and ϵ_{ij} is the error term for unit i at t_{ij} .

Parameter Structure in NLMM

- ▶ α_i is a $v \times 1$ linear predictor for subject i .
- ▶ Combines fixed effects and random effects:

$$\alpha_i = X_i\theta + Z\mathbf{a}_i.$$

- ▶ Conditional mean:

$$\mathbb{E}(y_{ij} \mid \mathbf{a}_i) = g(t_{ij}; X_i\theta + Z\mathbf{a}_i).$$

Covariate design matrix X_i :

- ▶ Links the basic parameters γ to subject-level covariates $\mathbf{x}_{p \times 1}$.
- ▶ Structure allows some parameters to depend on covariates, and others not.

Random Effects and Likelihood

- ▶ Random effects:

$$\mathbf{a}_i = (a_{i1}, \dots, a_{iq})^\top \text{ for } i = 1, \dots, s.$$

$$\mathbf{a}_i \sim N(\mathbf{0}, \text{Diag}(\sigma_1^2, \dots, \sigma_q^2)).$$

- ▶ Z selects which parameters receive random effects.
- ▶ The data for NLMM are denoted by $\{y_{ij}, t_{ij}, \mathbf{x}_i\}$, $i = 1, \dots, s$ and $j = 1, \dots, n_i$.

Random Effects and Likelihood

- The likelihood function is then

$$L(\psi) = \prod_{i=1}^s \int_{-\infty}^{\infty} \cdots \int_{-\infty}^{\infty} \left(\prod_{j=1}^{n_i} \frac{1}{\sigma} \phi_{\mathbf{N}}(w_{ij}) \right) \\ \times \left(\prod_{l=1}^q \frac{1}{\sigma_l} \phi_{\mathbf{N}}(a_{il}/\sigma_l) \right) da_{i1} \cdots da_{iq}, \quad (18)$$

where $w_{ij} = [y_{ij} - g(t_{ij}; \alpha_i)]/\sigma$.

Inference

- ▶ Maximizing (18) with respect to ψ produces the MLE values $\hat{\psi}$ via the marginal approach.
- ▶ Maximizing (18) can be challenging unless the dimension of $\mathbf{a}_i$ is small, particularly for the numerical integration.
- ▶ Numerical approximation methods are used.
- ▶ The Bayesian method can also be used.